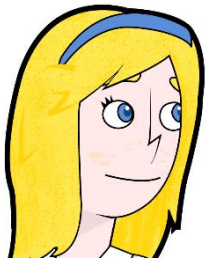
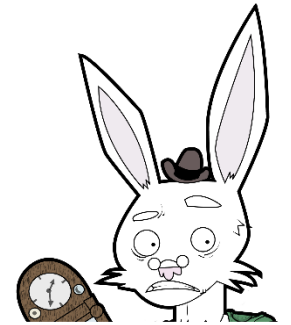


Data Privacy and Security

Chapter 3: Post-Quantum Cryptography



Prof. Daniele Venturi
Dipartimento di Informatica



SAPIENZA
UNIVERSITÀ DI ROMA

Master's Degree in Data Science

The Quantum Threat

- An algorithm by Shor solves the factoring and discrete logarithm problems in **polynomial-time** on a **quantum** machine
 - The algorithm requires an **ideal** quantum Turing machine
 - Factoring a 1024-bit integer requires **2050** logical **qubits** and a quantum circuit with **billions** of quantum gates
 - Despite recent progress on quantum computation, current implementations can only factor **tiny numbers** (e.g., 15 and 21)
- Nevertheless, the NIST started in 2017 a process to solicit, evaluate, and standardize **quantum-resistant** cryptography
 - The selected algorithms were announced in 2022
 - Most of these algorithms are based on **lattices**

From Period Finding to Factoring

- Shor's algorithm solves the **period finding problem**
- Let $n = p \cdot q$ for primes p, q ; pick **random** $x \in \mathbb{Z}_n^*$ (i.e., **coprime** with n)

$$x^0 \equiv 1 \pmod{n}; x \pmod{n}; \dots; x^r \pmod{n}$$

- Period: The **smallest** $0 < r < n$ s.t. $x^r \equiv 1 \pmod{n}$
 - Can show that w.p. $\geq 1/2$, the period r is **odd** and $x^{r/2} \pm 1$ is **not a multiple** of n

$$\begin{aligned} x^r \equiv 1 \pmod{n} &\Leftrightarrow (x^{r/2})^2 \equiv 1 \pmod{n} \Leftrightarrow \\ (x^{r/2} + 1)^2 (x^{r/2} - 1)^2 &\equiv 0 \pmod{n} \Leftrightarrow (x^{r/2} + 1)^2 (x^{r/2} - 1)^2 = k \cdot n \end{aligned}$$

- Since $k > 0$, both $\gcd(x^{r/2} \pm 1, n)$ are **non-trivial factors** of n

What's the Rush?

- Big quantum computers won't be available for **many years**
 - If **ever**...
 - Can't we just wait?
- Better safe than sorry
 - **Harvesting attacks**: Store today's keys/ciphertexts to break later
 - **Rewrite history**: Forge signatures for old keys
 - Deploying new cryptography **at scale** requires 10+ years

Lattices

Basic Example

- Consider the **modular equation**

$$26x + y \equiv 0 \pmod{43}$$

- A few **solutions** are: $(0,0)$, $(-1,26)$, $(0,43)$, $(1,17)$
- Are there **short solutions** (e.g., according to the $\|\cdot\|_\infty$ -norm)?
 - The solution $(0,0)$ is **short**, but uninteresting; are there **non-zero short solutions**?
- Let $\mathcal{L} = \{(x, y) \in \mathbb{Z}^2 : 26x + y \equiv 0 \pmod{43}\}$ be the set of **all solutions**
 - Note that if $(x_1, y_1), (x_2, y_2)$ are solutions, also $(-x_1, -y_1)$ and $(x_1 + x_2, y_1 + y_2)$ are solutions
 - Moreover, (x, y) is a solution iff $(x, y) = a(1, -26) + b(0, 43)$
 - We call $\mathbf{B} = \begin{pmatrix} 1 & 0 \\ -26 & 43 \end{pmatrix}$ a **basis** of the **lattice** $\mathcal{L}$

Multiple Dimensions

- Let's generalize the previous example to n **dimensions**
- For $q \geq 2$ and $a_1, \dots, a_n \in \mathbb{Z}_q$ consider:

$$\mathcal{L} = \left\{ \mathbf{z} \in \mathbb{Z}^n : \langle \mathbf{a}, \mathbf{z} \rangle = \sum_{i=1}^n a_i z_i \equiv 0 \pmod{q} \right\}$$

- Again, we can **efficiently** find a **basis** $\mathbf{B} = (\mathbf{b}_1, \dots, \mathbf{b}_n)$ for $\mathcal{L}$
 - Assume for simplicity q is a prime and that $a_1 \neq 0$
 - It's easy to check that $\mathbf{b}_1 = (q, 0, \dots, 0)$ and $\mathbf{b}_i = (-a_1^{-1}a_i, 0, \dots, 1, \dots, 0)$ is a basis

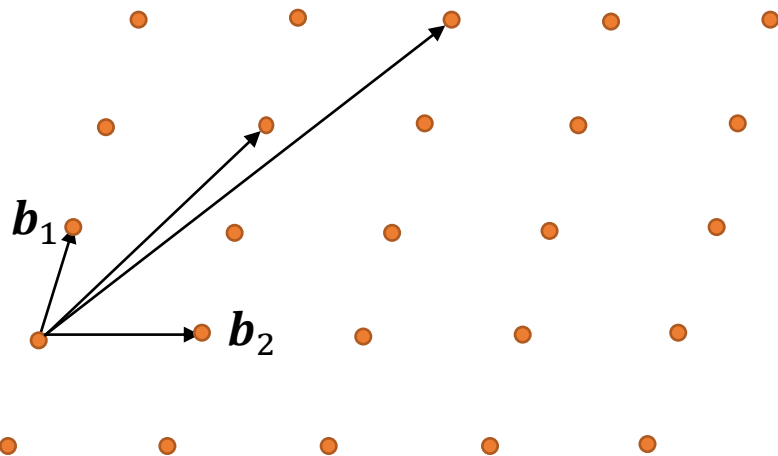
Existence of Short Solutions

- **Theorem:** For any $q \geq 2$ and any $\mathbf{a} \in \mathbb{Z}^n$ there is a **non-zero** vector $\mathbf{z} \in \mathbb{Z}^n$ such that $\langle \mathbf{a}, \mathbf{z} \rangle = 0 \pmod{q}$ and $\|\mathbf{z}\|_\infty \leq q^{1/n} + 1 \approx q^{1/n}$
 - Take the set of **all vectors** $\mathbf{z} \in \mathbb{Z}^n$ s.t. $\|\mathbf{z}\|_\infty \leq r = (q^{1/n} + 1)/2$
 - Consider the set of all **dot products** $\mathcal{S} = \{\langle \mathbf{a}, \mathbf{z} \rangle \pmod{q} : \|\mathbf{z}\|_\infty \leq r\}$
 - Clearly, $|\mathcal{S}| \leq q$ and there are $(2\lfloor r \rfloor + 1)^n > (2r - 1)^n = q$ vectors in $\mathbb{Z}^n$ with norm **at most** r
 - There must exist $\mathbf{z}_1, \mathbf{z}_2$ with $\mathbf{z}_1 \neq \mathbf{z}_2$ and $\|\mathbf{z}_1\|_\infty, \|\mathbf{z}_2\|_\infty \leq r$ s.t. $\langle \mathbf{a}, \mathbf{z}_1 \rangle = \langle \mathbf{a}, \mathbf{z}_2 \rangle \pmod{q}$
 - This means $\langle \mathbf{a}, \mathbf{z}_1 - \mathbf{z}_2 \rangle = 0 \pmod{q}$ and $\mathbf{z} = \mathbf{z}_1 - \mathbf{z}_2 \in \mathcal{L}$ with $\|\mathbf{z}\|_\infty \leq 2r$

What is a Lattice?

- Simply, a set of points in a **high-dimensional** space
 - Arranged **periodically**
- Formally, take n **linearly independent** vectors $\mathbf{B} = (\mathbf{b}_1, \dots, \mathbf{b}_n)$ in $\mathbb{R}^n$ and consider all **integer** combinations

$$\mathcal{L}(\mathbf{B}) = \left\{ \sum_{i=1}^n a_i \mathbf{b}_i : a_1, \dots, a_n \in \mathbb{Z} \right\}$$



- We call $(\mathbf{b}_1, \dots, \mathbf{b}_n)$ a **basis**
- The same lattice may have **different equivalent** basis
 - Again, we are interested in **short vectors** belonging to the lattice

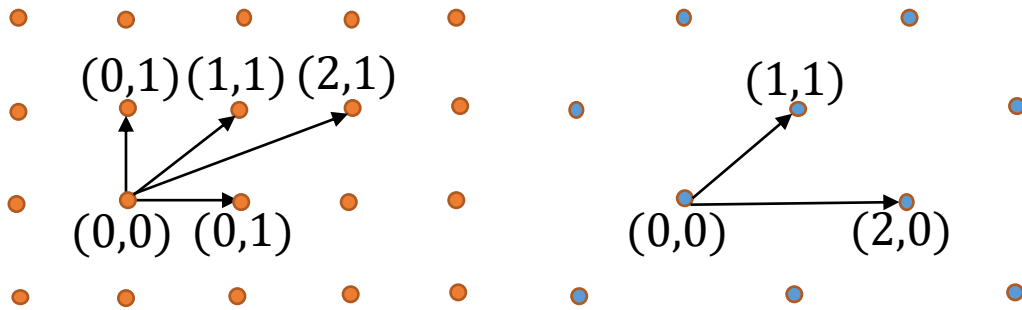
History

- **Geometric** objects with rich mathematical structure
- Considerable **mathematical interest** starting from Gauss (1801), Hermite (1850), and Minkowski (1896)



- Many **interesting applications** (cryptanalysis, factoring rational polynomials, finding integer relations, ...)

Equivalent Bases



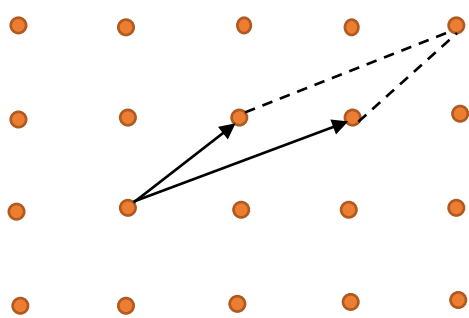
- **Equivalent** bases:
 - Permute vectors (i.e., $\mathbf{b}_i \leftarrow \mathbf{b}_j$)
 - Negate vectors (i.e., $\mathbf{b}_i \leftarrow (-\mathbf{b}_i)$)
 - Add integer multiple of another vector (i.e., $\mathbf{b}_i \leftarrow \mathbf{b}_i + k \cdot \mathbf{b}_j, k \in \mathbb{Z}$)
- **Theorem:** Two bases $\mathbf{B}_1, \mathbf{B}_2$ are **equivalent** iff $\mathbf{B}_2 = \mathbf{B}_1 \cdot \mathbf{U}$
 - $\mathbf{U}$ **unimodular** (i.e., integer matrix with $\det(\mathbf{U}) = \pm 1$)

Equivalent Bases

- Let $\mathbf{B}_1 = \mathbf{B}_2 \cdot \mathbf{U}$
 - If $\mathbf{U}$ is **unimodular**, so is $\mathbf{U}^{-1}$ and $\mathbf{B}_2 = \mathbf{B}_1 \cdot \mathbf{U}^{-1}$
 - Hence, $\mathcal{L}(\mathbf{B}_1) = \{\mathbf{B}_1 \cdot \mathbf{x} : \mathbf{x} \in \mathbb{Z}^n\} \subseteq \mathcal{L}(\mathbf{B}_2)$ and $\mathcal{L}(\mathbf{B}_2) \subseteq \mathcal{L}(\mathbf{B}_1)$ which means $\mathcal{L}(\mathbf{B}_1) = \mathcal{L}(\mathbf{B}_2)$
- Let $\mathbf{B}_1 = \mathbf{B}_2 \cdot \mathbf{W}$ and $\mathbf{B}_2 = \mathbf{B}_1 \cdot \mathbf{V}$ for **integer matrices** $\mathbf{V}, \mathbf{W}$
 - Hence, $\mathbf{B}_1 = \mathbf{B}_1 \cdot \mathbf{V} \cdot \mathbf{W}$ or $\mathbf{B}_1 \cdot (\mathbf{I} - \mathbf{V} \cdot \mathbf{W}) = \mathbf{0}$
 - Since the vectors in $\mathbf{B}_1$ are **linearly independent**, we can multiply both sides by $\mathbf{B}_1^{-1}$ and thus $\mathbf{I} - \mathbf{V} \cdot \mathbf{W} = \mathbf{0}$
 - Thus, $\mathbf{V} \cdot \mathbf{W} = \mathbf{I}$ and $\det(\mathbf{V}) \cdot \det(\mathbf{W}) = \det(\mathbf{V} \cdot \mathbf{W}) = 1$
 - Since $\mathbf{V}, \mathbf{W}$ are **integer matrices** $\det(\mathbf{V}), \det(\mathbf{W}) \in \mathbb{Z}$ and $\det(\mathbf{V}) = \det(\mathbf{W}) \pm 1$

The Fundamental Region

- The **fundamental region** of a lattice corresponds to a **periodic tiling** of $\mathbb{R}^n$ by copies of some body
 - For instance, $[0,1)$ is a fundamental region of the integer lattice $\mathbb{Z}$, as every $x \in \mathbb{R}$ is in the unique translate $[x] + [0,1)$



- A lattice base yields a fundamental region called the **fundamental parallelepiped**

$$\mathcal{P}(\mathbf{B}) = \mathbf{B} \cdot [0,1)^n = \left\{ \sum_{i=1}^n c_i \cdot \mathbf{b}_i : c_i \in [0,1) \right\}$$

- Clearly, different basis generate different fundamental parallelepipeds
 - Note that the parallelepiped associated to a **basis** does **not contain** any lattice point, while that associated to vectors not forming a basis does
 - This is not a coincidence!

The Fundamental Region

- **Theorem:** Let $\mathcal{L}$ be a lattice and $\mathbf{b}_1, \dots, \mathbf{b}_n \in \mathbb{R}^n$ be **linearly independent** vectors in $\mathcal{L}$; then, $\mathbf{B} = (\mathbf{b}_1, \dots, \mathbf{b}_n)$ form a basis iff $\mathcal{P}(\mathbf{B}) \cap \mathcal{L} = \{\mathbf{0}\}$
- Let $\mathbf{B}$ be a basis
 - Consider $\mathbf{a} = \sum_i x_i \mathbf{b}_i \in \mathcal{L}(\mathbf{B}) \cap \mathcal{P}(\mathbf{B})$
 - Since $\mathbf{a} \in \mathcal{L}(\mathbf{B})$ we have $x_i \in \mathbb{Z}$, but $\mathbf{a} \in \mathcal{P}(\mathbf{B})$ requires $x_i < 1$; thus $x_i = 0$
- Let $\mathcal{P}(\mathbf{B}) \cap \mathcal{L} = \{\mathbf{0}\}$
 - Pick any $\mathbf{a} \in \mathcal{L}$ and write $\mathbf{a} = \sum_i x_i \mathbf{b}_i$ with $x_i \in \mathbb{R}$
 - Consider $\mathbf{a}' = \sum_i \lfloor x_i \rfloor \mathbf{b}_i$; since $\lfloor x_i \rfloor$ is integer, $\mathbf{a}' \in \mathcal{L}(\mathbf{B}) \subseteq \mathcal{L}$ and $\mathbf{a} - \mathbf{a}' \in \mathcal{L}$
 - But $\mathbf{a} - \mathbf{a}' = \sum_i (x_i - \lfloor x_i \rfloor) \cdot \mathbf{b}_i \in \mathcal{P}(\mathbf{B})$ as well, and thus $\mathbf{a} - \mathbf{a}' = \mathbf{0}$
 - Since $\mathbf{b}_1, \dots, \mathbf{b}_n \in \mathbb{R}^n$ are **linearly independent** it must be $x_i - \lfloor x_i \rfloor = 0$
 - Thus, $x_i \in \mathbb{Z}$ and $\mathbf{a} \in \mathcal{L}(\mathbf{B})$ which means $\mathcal{L} \subseteq \mathcal{L}(\mathbf{B})$
 - Since, clearly, it is also true that $\mathcal{L}(\mathbf{B}) \subseteq \mathcal{L}$, we conclude $\mathcal{L}(\mathbf{B}) = \mathcal{L}$

Determinant

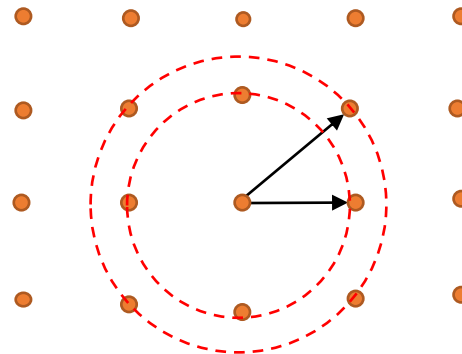
- The **determinant** of a lattice $\mathcal{L}(\mathbf{B})$ is $\det(\mathcal{L}) = |\det(\mathbf{B})|$
- Note that this is well defined, as for every **unilateral** U

$$|\det(\mathbf{B} \cdot \mathbf{U})| = |\det(\mathbf{B}) \cdot \det(\mathbf{U})| = \det(\mathbf{B})$$

- The determinant corresponds to the **volume** of the fundamental parallelepiped
 - The determinant is the **reciprocal** of the **density** (i.e., a big determinant means a sparse lattice)
 - Moreover, the volume is the **same** for **every** fundamental region

Successive Minima

- Let $\lambda_1(\mathcal{L})$ be the length of a **shortest non-zero** vector in $\mathcal{L}$
 - Usually, in terms of the **Euclidean** norm $\|x\| = \sqrt{\sum_i x_i^2}$
 - The shortest vector is **never unique**, as for every $x \in \mathcal{L}$ also $-x \in \mathcal{L}$
- More generally, $\lambda_k(\mathcal{L})$ denotes the radius of the ball containing k **linearly independent** vectors
 - For $k = n$ the ball contains a basis of the entire space



Upper Bounding λ_1

- Since the **determinant** of a lattice $\mathcal{L}$ is **inversely proportional** to the density, we should be able to upper bound $\lambda_1(\mathcal{L})$ using $\det(\mathcal{L})$
- One can easily show that $\lambda_1(\mathcal{L}) \leq \det(\mathcal{L})$
- However, this does not scale well
 - The lattice $k\mathcal{L}$ obtained by scaling every vector in $\mathcal{L}$ by k should have a **short vector** of length $k \cdot \lambda_1(\mathcal{L})$
 - But $\det(k\mathcal{L}) = k^n \det(\mathcal{L})$
- We will now show a **much stronger** bound
- **Theorem (Minkowski)**: For any (full-rank) lattice $\mathcal{L}$, it holds that

$$\lambda_1(\mathcal{L}) \leq \sqrt{n} \cdot \det(\mathcal{L})^{1/n}$$

Proof of Minkowski's First Theorem

- **Lemma (Blichfeld)**: For any lattice $\mathcal{L}$ and set $\mathcal{S} \in \mathbb{R}^n$ with $\text{vol}(\mathcal{S}) > \det(\mathcal{L})$, $\exists$ **distinct** $\mathbf{x}, \mathbf{y} \in \mathcal{S}$ s.t. $\mathbf{x} - \mathbf{y} \in \mathcal{L}$
- Let $\mathbf{B}$ be a basis for $\mathcal{L}$ and define $f: \mathbb{R}^n \rightarrow \mathcal{P}(\mathbf{B})$ such that

$$f\left(\sum_i x_i \cdot \mathbf{b}_i\right) = \sum_i (x_i - \lfloor x_i \rfloor) \cdot \mathbf{b}_i$$

- Note that $\mathbf{x} - f(\mathbf{x}) = \sum_i x_i \cdot \mathbf{b}_i - f(\sum_i x_i \cdot \mathbf{b}_i) = \sum_i \lfloor x_i \rfloor \cdot \mathbf{b}_i \in \mathcal{L}$
- **Case 1**: Assume $\exists$ **distinct** $\mathbf{x}, \mathbf{y} \in \mathcal{S}$ s.t. $f(\mathbf{x}) = f(\mathbf{y})$
 - Then, $\mathbf{x} - \mathbf{y} = (\mathbf{x} - f(\mathbf{x})) - (\mathbf{y} - f(\mathbf{y})) \in \mathcal{L}$

Proof of Minkowski's First Theorem

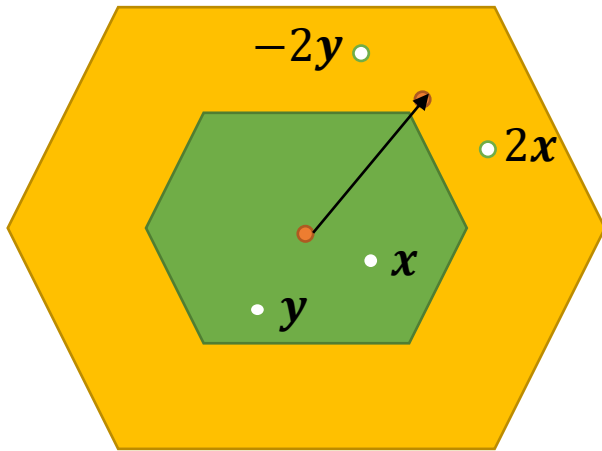
- **Lemma (Blichfeldt)**: For any lattice $\mathcal{L}$ and set $\mathcal{S} \in \mathbb{R}^n$ with $\text{vol}(\mathcal{S}) > \det(\mathcal{L})$, $\exists$ **distinct** $\mathbf{x}, \mathbf{y} \in \mathcal{S}$ s.t. $\mathbf{x} - \mathbf{y} \in \mathcal{L}$
- **Case 2**: Assume **no collisions** and let $\mathcal{S}_x = \mathcal{S} \cap (\mathbf{x} + \mathcal{P}(\mathbf{B}))$ for $\mathbf{x} \in \mathcal{L}(\mathbf{B})$
 - So, $\mathcal{S} = \bigcup_{\mathbf{x} \in \mathcal{L}(\mathbf{B})} \mathcal{S}_x$ and $\text{vol}(\mathcal{S}) = \sum_{\mathbf{x} \in \mathcal{L}(\mathbf{B})} \text{vol}(\mathcal{S}_x)$
 - Moreover, $\tilde{\mathcal{S}}_x = \mathcal{S}_x - \mathbf{x} = (\mathcal{S} - \mathbf{x}) \cap \mathcal{P}(\mathbf{B}) \subseteq \mathcal{P}(\mathbf{B})$ and $\text{vol}(\mathcal{S}_x) = \text{vol}(\tilde{\mathcal{S}}_x)$
 - Since there are **no collisions**, for all $\mathbf{x}, \mathbf{y} \in \mathcal{S}$ we have $\tilde{\mathcal{S}}_x \cap \tilde{\mathcal{S}}_y = \emptyset$
 - Then, $\text{vol}(\mathcal{S}) = \sum_{\mathbf{x} \in \mathcal{L}(\mathbf{B})} \text{vol}(\tilde{\mathcal{S}}_x) = \text{vol}(\bigcup_{\mathbf{x} \in \mathcal{L}(\mathbf{B})} \tilde{\mathcal{S}}_x) \leq \text{vol}(\mathcal{P}(\mathbf{B})) = \det(\mathcal{L})$
 - This **contradicts** the assumption that $\text{vol}(\mathcal{S}) > \det(\mathcal{L})$

Proof of Minkowski's First Theorem

- **Definition:** A set $\mathcal{S} \in \mathbb{R}^n$ is **convex** if for all **distinct** $\mathbf{x}, \mathbf{y} \in \mathcal{S}$ it holds that $\alpha \mathbf{x} + (1 - \alpha) \mathbf{y} \in \mathcal{S}$ for ever $\alpha \in [0,1]$
 - That is, if we take any two points from a convex set, any point that lies on the **straight line** between the two points must also be in the set
- **Definition:** A set $\mathcal{S} \in \mathbb{R}^n$ is **centrally symmetric** if for all $\mathbf{x} \in \mathcal{S}$ it holds that $-\mathbf{x} \in \mathcal{S}$
- **Theorem:** For any lattice $\mathcal{L}$ and **convex, centrally symmetric**, set $\mathcal{S}$ with $\text{vol}(\mathcal{S}) > 2^n \det(\mathcal{L})$, there exists a **non-zero** lattice point in $\mathcal{S}$

Proof of Minkowski's First Theorem

- **Theorem:** For any lattice $\mathcal{L}$ and **convex, centrally symmetric**, set $\mathcal{S}$ with $\text{vol}(\mathcal{S}) > 2^n \det(\mathcal{L})$, there exists a **non-zero** lattice point in $\mathcal{S}$



- Let $\mathcal{S}/2 = \{x: 2x \in \mathcal{S}\}$ with $\text{vol}(\mathcal{S}/2) = 2^{-n} \cdot \text{vol}(\mathcal{S}) > \det(\mathcal{L})$
- By **Blichfeldt** $\exists$ **distinct** $x, y \in \mathcal{S}/2$ s.t. $x - y \in \mathcal{L}$
- By symmetry, $2x, -2y \in \mathcal{S}$; by convexity $x - y = \frac{2x - 2y}{2} \in \mathcal{S}$

- **Corollary:** For every $\mathcal{L}$, we have that $\lambda_1(\mathcal{L}) \leq \sqrt{n} \cdot \det(\mathcal{L})^{1/n}$
 - Let $\mathcal{S} = \mathcal{B}_n(0, \lambda_1)$ be the **open ball** of radius λ_1 centered on the origin
 - Since $\mathcal{B}_n(0, r)$ contains the **hypercube** $\mathcal{C} = \{x \in \mathbb{R}^n: |x_i| < r/\sqrt{n}\}$ we have that $\text{vol}(\mathcal{B}_n(0, r)) \geq (2r/\sqrt{n})^n$
 - By the above theorem, $(2\lambda_1/\sqrt{n})^n \leq \text{vol}(\mathcal{B}_n(0, \lambda_1)) \leq 2^n \det(\mathcal{L})$

Minkowski's Second Theorem

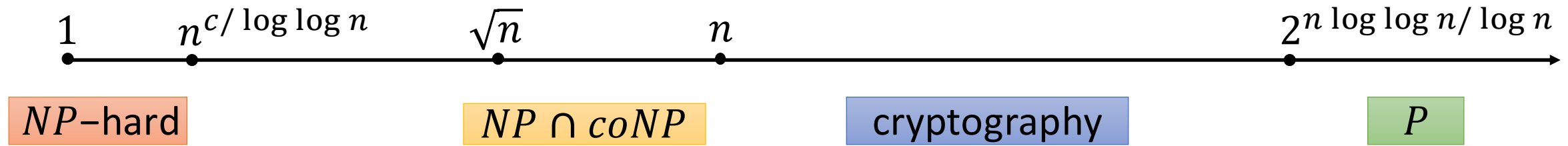
- Recall that $\lambda_k(\mathcal{L})$ denotes the radius of the ball containing k **linearly independent** vectors
- **Theorem:** For any lattice $\mathcal{L}$,

$$\left(\prod_i \lambda_i\right)^{1/n} \leq \sqrt{n} \cdot (\det(\mathcal{L}))^{1/n}$$

Hard Problems

- **SVP $_{\gamma}$** : Given $\mathbf{B}$, **find** a vector in $\mathcal{L}(\mathbf{B})$ with length $\leq \gamma \cdot \lambda_1(\mathcal{L}(\mathbf{B}))$
- **GapSVP $_{\gamma}$** : Given $\mathbf{B}$, **decide** if $\lambda_1(\mathcal{L}(\mathbf{B}))$ is ≤ 1 or $> \gamma$
- **SIVP $_{\gamma}$** : Given $\mathbf{B}$, **find** n **linearly independent** vectors in $\mathcal{L}(\mathbf{B})$ with length $\leq \gamma \cdot \lambda_n(\mathcal{L}(\mathbf{B}))$
- **CVP $_{\gamma}$** : Given $\mathbf{B}$ and $\mathbf{v} \in \mathbb{R}^n$, find a **lattice point** that is at most γ times **further** away than the **closest** lattice point
 - It is known that **SVP $_{\gamma} \leq$ CVP $_{\gamma}$**
- **BDD**: Find the **closest** lattice point, knowing that $\mathbf{v}$ is **already close**

General Hardness Results



- Exact algorithms take time 2^n
- **Polynomial-time** algorithm for gap $\gamma = 2^n \log \log n / \log n$
- No better **quantum** algorithm known
- **NP hardness** for gap $\gamma = n^c / \log \log n$
 - For cryptographic applications, we need $\gamma = \Omega(n)$
 - Not believed to be NP -hard for $\gamma = \sqrt{n}$

Small Integer Solution Problem

- Fix **dimension** n , and **modulus** q (e.g., $q \approx n^2$)
- Given **random** vectors $\mathbf{a}_1, \dots, \mathbf{a}_m \in \mathbb{Z}_q^n$, find **non-zero small** $z_1, \dots, z_m \in \mathbb{Z}$ such that

$$z_1 \cdot \mathbf{a}_1 + z_2 \cdot \mathbf{a}_2 + \dots + z_m \cdot \mathbf{a}_m = \mathbf{0} \quad \text{in } \mathbb{Z}_q^n$$

- Observations:
 - Trivial if the size of the z_i 's is **not restricted** (Gaussian elimination)
 - Equivalently, find **non-zero short** $\mathbf{z} \in \mathbb{Z}^m$ s.t. $\mathbf{A} \cdot \mathbf{z} = \mathbf{0} \in \mathbb{Z}_q^n$

SIS as a Lattice Problem

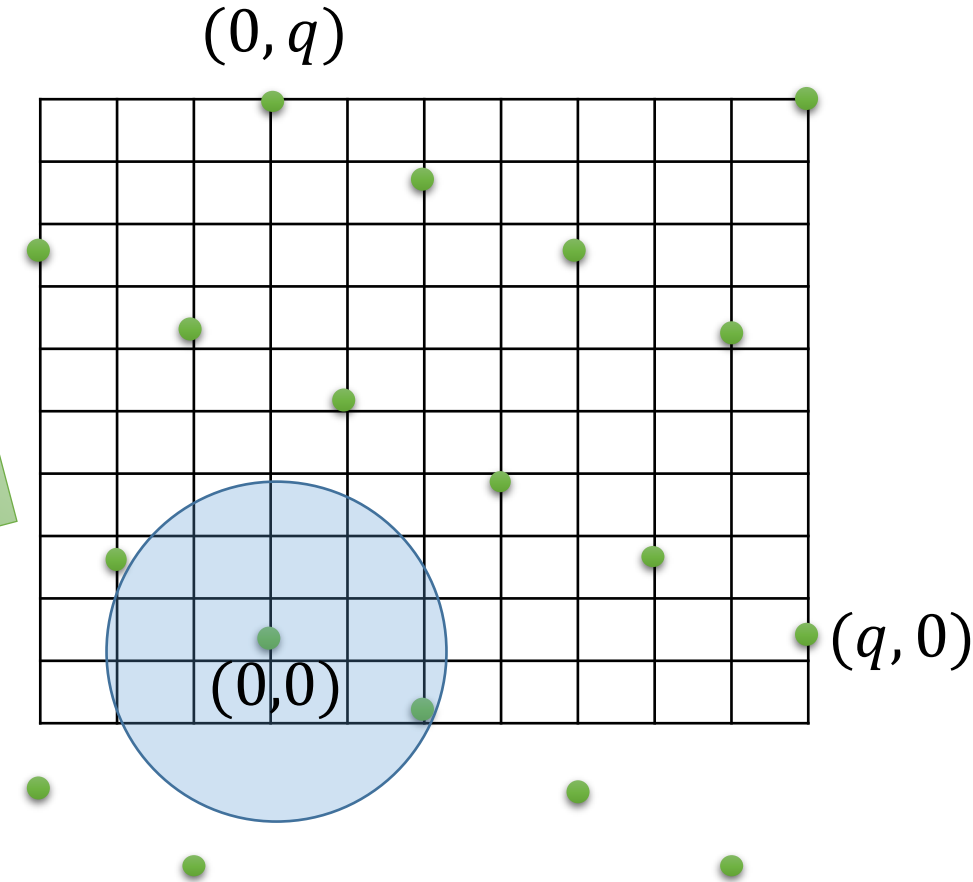
- Matrix $A = (a_1, \dots, a_m) \in \mathbb{Z}_q^{n \times m}$
 $\mathcal{L}^\perp(A) = \{z \in \mathbb{Z}^m : A \cdot z = 0\}$

Find **short** ($\|z\| \leq \beta \ll q$)
solutions for **random** A

- Theorem:** For **any** n -dimensional lattice, it holds that:

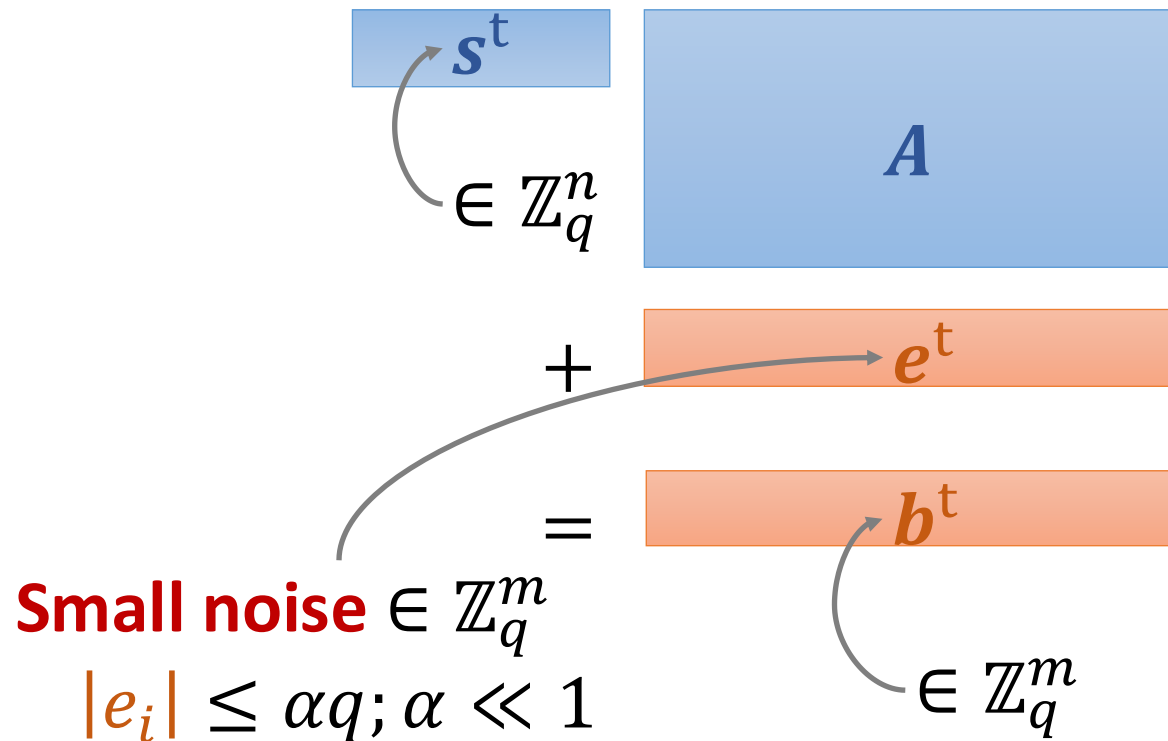
$$\text{GapSVP}_{\beta\sqrt{n}}, \text{SIVP}_{\beta\sqrt{n}} \leq \text{SIS}_\beta$$

- Also true for any lattice **coset** $\mathcal{L}_u^\perp(A) = \{z \in \mathbb{Z}^m : A \cdot z = u\} = u + \mathcal{L}^\perp(A)$ (i.e., **inhomogeneous** SIS)



Learning with Errors

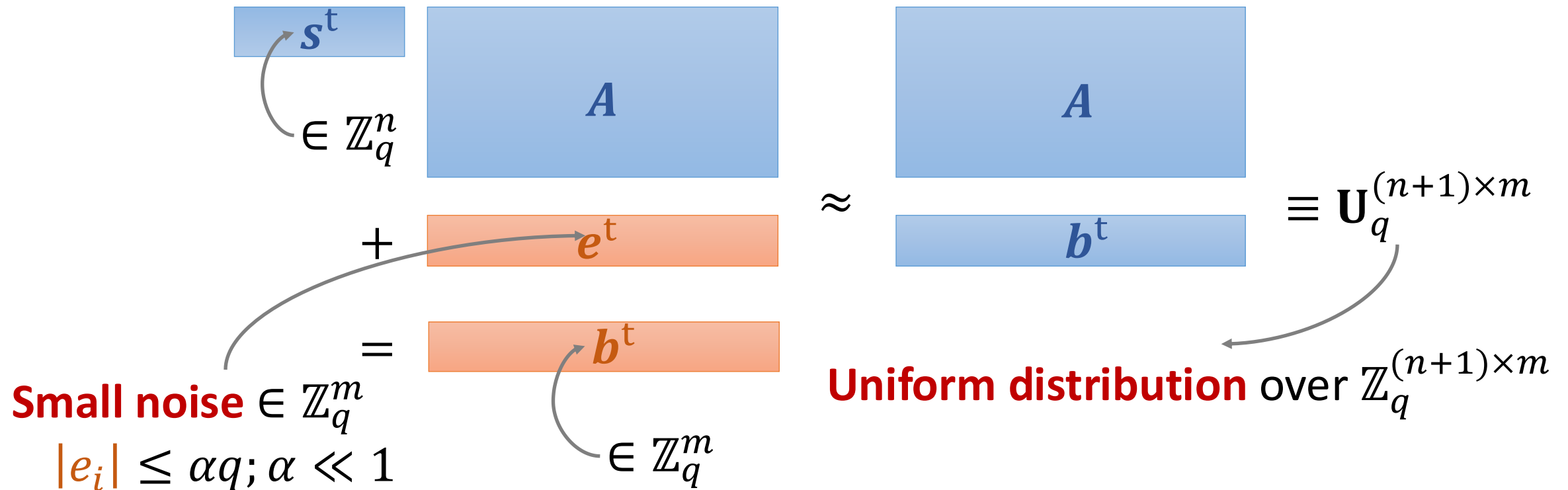
- Dimension n , modulus $q > 2$, **noise** distribution χ
- **Find** $\mathbf{s} \in \mathbb{Z}_q^n$ given m **noisy random** inner product equations



- Trivial **without** noise
- **Gaussian** distribution over $\mathbb{Z}$, with std deviation $\geq \sqrt{n}$ and $\ll q$
 - Rate parameter $\alpha \ll 1$
- Need $\alpha q > \sqrt{n}$ for **worst-case hardness** and because there is an $\exp((\alpha q)^2)$ -time attack

Decisional LWE

- **Distinguish** the matrix A and the vector b from random (A, b)
 - Decisional LWE is **equivalent** to Search LWE



LWE as a Lattice Problem

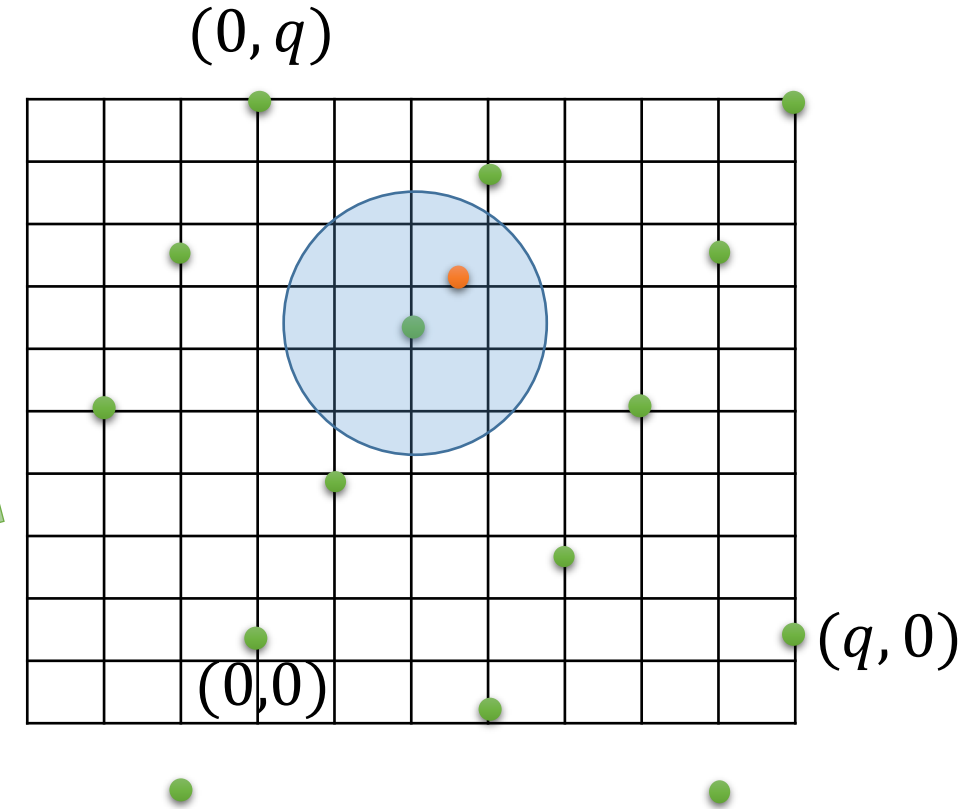
- Matrix $A = (a_1, \dots, a_m) \in \mathbb{Z}_q^{n \times m}$
 $\mathcal{L}(A) = \{z \in \mathbb{Z}^m : z^t = s^t \cdot A\}$

LWE is BDD on $\mathcal{L}(A)$: Given
 $b^t \approx z^t = s^t \cdot A$ find z

- Theorem:** For **any** n -dimensional lattice, it holds that:

$$\text{GapSVP}_{\alpha n}, \text{SIVP}_{\alpha n} \leq \text{LWE}$$

- Quantum** reduction for **broad** parameters
- Classical** reduction for **restricted** parameters (e.g., $q \approx 2^n$)



Hardness of LWE

- More formally define the **LWE distribution** as

$$\text{LWE}[n, m, q, \chi] = \left\{ (A, b) : \begin{array}{l} A \leftarrow \mathbb{Z}_q^{n \times m}; s \leftarrow \mathbb{Z}_q^n; \\ e \leftarrow \chi^m; b^t = [s^t \cdot A + e^t]_q \end{array} \right\}$$

- Parameters:
 - $\alpha = 1/\text{poly}(n)$ or $\alpha = 2^{-n^\epsilon}$ (**stronger** assumption as α **decreases**)
 - $m = \Theta(n \log q)$ or $m = \text{poly}(n)$ (**stronger** assumption as m **increases**)
 - $q = 2^{n^\epsilon}$ or $q = \text{poly}(n)$ (**stronger** assumption as q **increases**)
 - Noise distribution χ such that $\mathbb{P}[|e| > \alpha q : e \leftarrow \chi] \leq \text{negl}(n)$

Simple Properties

- Check a **candidate** solution $\mathbf{t} \in \mathbb{Z}_q^n$
 - Test if all $b - \langle \mathbf{t}, \mathbf{a} \rangle$ are small
 - If $\mathbf{t} \neq \mathbf{s}$, then $b - \langle \mathbf{t}, \mathbf{a} \rangle = \langle \mathbf{s} - \mathbf{t}, \mathbf{a} \rangle + e$ is **well-spread** in $\mathbb{Z}_q$
- **Shift** the secret by any $\mathbf{r} \in \mathbb{Z}_q^n$
 - Given $(\mathbf{a}, b = \langle \mathbf{s}, \mathbf{a} \rangle + e)$, output $(\mathbf{a}, b' = b + \langle \mathbf{r}, \mathbf{a} \rangle = \langle \mathbf{s} + \mathbf{r}, \mathbf{a} \rangle + e)$
 - Using **random** $\mathbf{r}$ yields a random **self-reduction**
 - **Amplification** of success probabilities (i.e., **non-negligible** success probability for **random** $\mathbf{s} \in \mathbb{Z}_q^n$ implies **overwhelming** success probability for **every** $\mathbf{s} \in \mathbb{Z}_q^n$)
- **Multiple** secrets: $(\mathbf{a}, b_1 = \langle \mathbf{s}_1, \mathbf{a} \rangle + e_1, \dots, \langle \mathbf{s}_t, \mathbf{a} \rangle + e_t)$ indistinguishable from **random** $(\mathbf{a}, b_1, \dots, b_t)$

Search/Decision Equivalence

- Suppose we are given an oracle that **perfectly distinguishes** pairs $(a, b = \langle s, a \rangle + e)$ from random (a, b)
- To find s_1 , it suffices to **test** if $s_1 = 0$
 - Because we can **shift** s_1 by $0, 1, \dots, q - 1$ (assuming $q = \text{poly}(n)$)
 - Then we can do the same for $s_2, \dots, s_n$
- The test: For each (a, b) , choose **random** $r \in \mathbb{Z}_q$ and invoke the oracle on pairs $(a' = a - (r, 0, \dots, 0), b)$
- Note that $b = \langle s, a' \rangle + s_1 \cdot r + e$
 - If $s_1 = 0$, then $b = \langle s, a' \rangle + e$ and the oracle **accepts**
 - If $s_1 \neq 0$, then b is **uniform** (assuming q **prime**) and the oracle **rejects**

LWE with Short Secrets

- **Theorem:** LWE is **no easier** if the secret is drawn from the **error distribution** χ
 - Intuition: Finding **e equivalent** to finding **s** (i.e., $b^t - e^t = s^t \cdot A$)
- **Transformation** from secret $s \in \mathbb{Z}_q^n$ to secret $\bar{e} \leftarrow \chi^n$
 - Draw samples to get $(\bar{A}, \bar{b}^t = s^t \cdot \bar{A} + \bar{e}^t)$ for square, invertible, $\bar{A}$
 - Transform each **additional** sample $(a, b = \langle s, a \rangle + e)$ to

$$a' = -\bar{A}^{-1} \cdot a, b' = b + \langle \bar{b}, a' \rangle = \langle \bar{e}, a' \rangle + e$$

- This maps **uniform** (a, b) to **uniform** (a', b') , and works for **decision** LWE too

LWE vs SIS

- SIS has **many** valid solutions, whereas LWE only has **one**
- **LWE $\leq$ SIS**
 - Given $\mathbf{z}$ such that $\mathbf{A} \cdot \mathbf{z} = \mathbf{0}$ from an SIS oracle, compute $\mathbf{b}^t \cdot \mathbf{z}$
 - Now, $\mathbf{b}^t \cdot \mathbf{z} = \mathbf{e}^t \cdot \mathbf{z}$ is **small** in the LWE case, whereas $\mathbf{b}^t \cdot \mathbf{z}$ is **well-spread** in case $\mathbf{b}^t$ is uniformly random
- What about the other direction?
 - Not known **in general**
 - True under **quantum reductions**

Efficiency of LWE/SIS

- Getting **one** random-looking scalar $b_i \in \mathbb{Z}_q$ requires an n -dimensional **inner product** mod q

$s^t \in \mathbb{Z}_q^n$

A

$+$

e^t

$=$

$b^t \in \mathbb{Z}_q^m$

Small noise $e^t \in \mathbb{Z}_q^m$
 $|e_i| \leq \alpha q; \alpha \ll 1$

- Can **amortize** each column a_i over **many secrets** s_j , but the latter still requires $\tilde{O}(n)$ work per scalar output
- Public keys are **rather large**, i.e. $> n^2$ time to encrypt/decrypt an n -bit message
- Can we do better?

Wishful Thinking...

$$\begin{array}{c} \text{blue bar} \\ s^t \end{array} \star \begin{array}{c} \text{blue bar} \\ a^t \end{array} + \begin{array}{c} \text{orange bar} \\ e^t \end{array} = \begin{array}{c} \text{orange bar} \\ b^t \end{array}$$

$\in \mathbb{Z}_q^d$

- Get d **pseudorandom** scalars from just one **cheap product** operation $\star$
- Replace $\mathbb{Z}_q^{d \times d}$ **chunks** with $\mathbb{Z}_q^d$

- **Main question:** How to define the product $\star$ so that (a, b) is **pseudorandom**
 - Requires care: **coordinate-wise** product **insecure** for **small** errors
- Let $\star$ be multiplication **in a polynomial ring**, e.g. $\mathbb{Z}_q^d[X]/(X^d + 1)$
 - **Fast** and **practical** with the FFT: $d \log d$ operations mod q
 - The same **ring structure** used in other cryptosystems (e.g., NTRU)

LWE over Rings/Modules

- Let $R = \mathbb{Z}[X]/(X^d + 1)$ for d a power of 2 and $R_q = R/qR$
 - Elements of R_q are degree $< d$ **polynomials** with coefficients mod q
 - Operations over R_q are **very efficient** using FFT-like algorithms
- **Search LWE**: Find secret vector of **polynomials** $\mathbf{s}$ in R_q^k given
 - Each equation is d **related equations** on a secret of dimension $n = d \cdot k$
 - LWE: $d = 1, k = n$
 - Ring-LWE: $d = n, k = 1$
 - Module-LWE: Interpolate
 - **Decision LWE**: Distinguish $(\mathbf{a}_i, \mathbf{b}_i)$ from uniform $(\mathbf{a}_i, \mathbf{b}_i)$ in $R_q^k \times R_q$

$$\mathbf{s}^t \star \mathbf{a}_i^t + \mathbf{e}_i^t = \mathbf{b}_i^t$$

$\mathbf{b}_i^t \in \mathbb{Z}_q^d$

Hardness of Ring/Module-LWE

- **Theorem**: For any $R = \mathcal{O}_K$

$$R^k\text{-GapSVP} \leq \text{search } R^k\text{-LWEdecision} \leq R^k\text{-LWE}$$

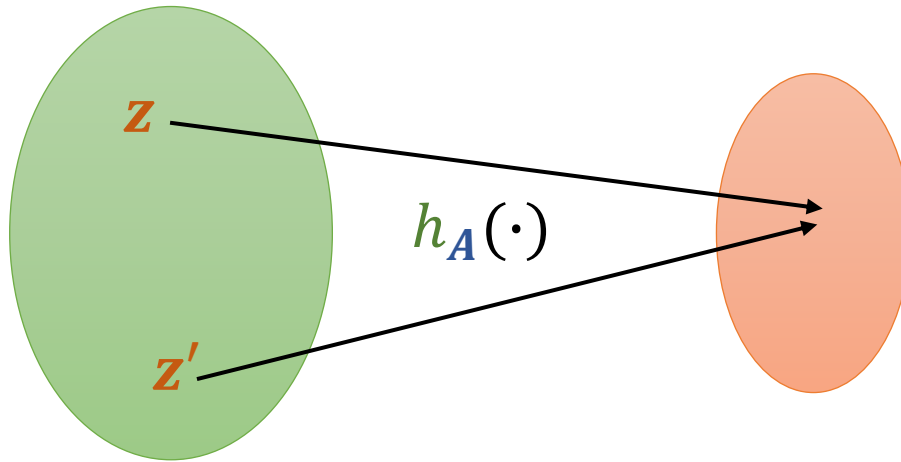
- Can we **dequantize** the worst-case/average-case reduction?
 - The **classical** GapSVP $\leq$ LWE reduction is of little use: for the relevant factors, GapSVP for **ideals** (i.e., $k = 1$) is **easy**
- How **hard** (or not) is **GapSVP** on **ideal/module lattices**?
 - For **polynomial approximation** no significant improvement versus general lattices (even for ideals)
 - For **subexponential approximation** we have better **quantum** algorithms for **ideals**, but not for $k > 1$
- **Reverse** reductions? Seems not **without** increasing k ...

Why Lattice-based Cryptography?

- **Provable** security
 - If scheme is not secure, one can solve hard mathematical problems
 - Not always happens in current implementations (e.g., RSA)
- **Worst-case** security
 - If scheme not secure, one can break **every** instance of lattice problems
 - Factoring and discrete log only guarantee **average-case** security
- Still **unbroken** by quantum algorithms
 - No progress over the last 50 years
 - But we don't really know!
- Efficiency
 - Mainly additions/multiplications, no modular exponentiations

Basic Cryptographic Applications

Collision-resistant Hash Functions



Collisions **exists inherently**, but are hard to find **efficiently**

- Given $A = (a_1, \dots, a_m)$, define $h_A: \{0,1\}^m \rightarrow \mathbb{Z}_q^n$

$$h_A(z_1, \dots, z_m) = a_1 \cdot z_1 + \dots + a_m \cdot z_m$$

- Set $m > n \log q$ in order to get **compression**
- A collision $a_1 \cdot z_1 + \dots + a_m \cdot z_m = a_1 \cdot z'_1 + \dots + a_m \cdot z'_m$ yields $a_1 \cdot (z_1 - z'_1) + \dots + a_m \cdot (z_m - z'_m) = 0$, with $z_m - z'_m \in \{-1, 0, 1\}$

Commitments

- Analogy: **lock** message in a box, give the box, keep the key
 - Later give the key to **open** the box
- Implementation:
 - **Randomized** function $\mathbf{Com}(x; r)$, where x is the message and r is the randomness
 - To **open** a commitment simply reveal (x, r)
- Security properties
 - Hiding: $\mathbf{Com}(x; r)$ **reveals nothing** on x
 - Binding: **Can't open** $\mathbf{Com}(x; r)$ to $x' \neq x$

Commitments

- Take two **random** SIS matrices A_1, A_2
- The **message** is $x \in \{0,1\}^m$ and the **randomness** is $r \in \{0,1\}^m$
- Commitment: $\mathbf{Com}(x; r) = f_{A_1, A_2}(x, r) = A_1 \cdot x + A_2 \cdot r$
 - **Hiding:** $A_2 \cdot r = f_{A_2}(r)$ is **statistically** close to **uniform** over $\mathbb{Z}_q^n$, and thus x is information-theoretically **hidden**
 - **Binding:** Finding (x, r) and (x', r') such that $\mathbf{Com}(x; r) = \mathbf{Com}(x'; r')$ directly contradicts the **collision resistance** of f_{A_1, A_2}

Leftover Hash Lemma

- **Theorem:** Let $\mathcal{H}$ be a family of **universal hash functions** with domain $\mathcal{D}$ and image $\mathcal{I}$. Then, for $x \leftarrow_{\$} \mathcal{D}$, $h \leftarrow_{\$} \mathcal{H}$, and $u \leftarrow_{\$} \mathcal{I}$:

$$\text{SD} \left((h, h(x)); (h, u) \right) \leq 1/2 \cdot \sqrt{|\mathcal{I}|/|\mathcal{D}|}$$

- Note that the function $h_A(\mathbf{r}) = [\mathbf{A} \cdot \mathbf{r}]_q$ is **universal**
 - As $\forall \mathbf{r}_1 \neq \mathbf{r}_2: \mathbb{P}_A[h_A(\mathbf{r}_1) = h_A(\mathbf{r}_2)] = \mathbb{P}_A[\mathbf{A} \cdot (\mathbf{r}_1 - \mathbf{r}_2) = \mathbf{0}] = q^{-n}$
- Hence, for $\mathbf{r} \leftarrow_{\$} \{0,1\}^m$, $\mathbf{A} \leftarrow_{\$} \mathbb{Z}_q^{n \times m}$, and $\mathbf{u} \leftarrow_{\$} \mathbb{Z}_q^n$, whenever $m = 2 + n \log q + 2n$:

$$\text{SD} \left((\mathbf{A}, [\mathbf{A} \cdot \mathbf{r}]_q); (\mathbf{A}, \mathbf{u}) \right) \leq 1/2 \cdot \sqrt{q^n/2^m} \leq 2^{-n}$$

Pseudorandom Functions

- Family $\mathcal{F} = \{F_s: \{0,1\}^k \rightarrow \mathcal{D}\}$ s.t. querying F_s , for **random** s , is indistinguishable from querying **random function** U



- Countless applications: **secret-key** encryption, message **authentication** codes, secure **identification**, ...

Constructing PRFs

- **Heuristically**: AES, etc.
 - Fast, secure against **known** cryptanalytic attacks, **not** provably secure
- From **any OWF** [GGM84]:
 - For **any** length-doubling **PRG** $G(s) = (G_0, G_1)$, let
$$F_s(x_1, \dots, x_k) = G_{x_k}(\dots G_{x_1}(s) \dots)$$
 - **Provably** secure
 - Inherently **sequential** (i.e., $\geq k$ iterations)
- From **any synthesizer** [NR95,NR97,NRR00]
 - **Low depth**: NC^1, NC^2 or TC^0 (i.e., $O(1)$ depth with **threshold** gates)
 - **Provably** secure

Synthesisers

- A **deterministic** function $S: \mathcal{D} \times \mathcal{D} \rightarrow \mathcal{D}$ such that for any polynomial m , and for **uniform** $a_1, \dots, a_m, b_1, \dots, b_m \in \mathcal{D}$

$$\{S(a_i, b_j)\} \approx \{U_{i,j}\}$$

Uniform distribution
over $\mathcal{D}^{m \times m}$

	b_1	b_2	...
a_1	$S(a_1, b_1)$	$S(a_1, b_2)$	
a_2	$S(a_2, b_1)$	$S(a_2, b_2)$	
...			

$\approx$

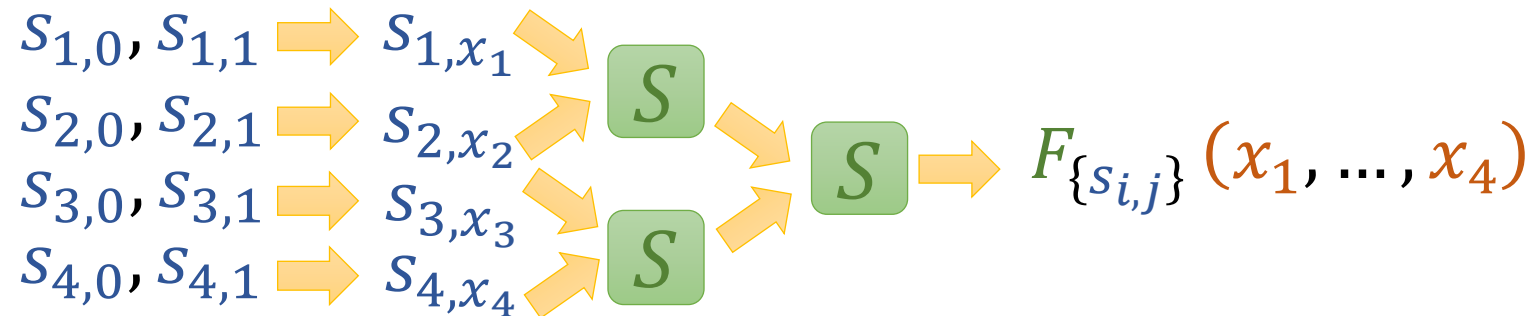
	b_1	b_2	...
a_1	$U_{1,1}$	$U_{1,2}$	
a_2	$U_{2,1}$	$U_{2,2}$	
...			

- An almost **length-squaring** PRG with **locality**

PRFs from Synthetisers

- **Base case:** One-bit PRF $F_{s_0, s_1}(x) = s_x \in \mathcal{D}$
- **Inductive step:** Given a k -bit PRF family $\mathcal{F} = \{F_s: \{0,1\}^k \rightarrow \mathcal{D}\}$ define $F_{s_L, s_R}: \{0,1\}^{2k} \rightarrow \mathcal{D}$

$$F_{s_L, s_R}(x_L, x_R) = S(F_{s_L}(x_L), F_{s_R}(x_R))$$



- **Security:** Every query to $F_{s_L}(x_L), F_{s_R}(x_R)$ defines **pseudorandom** inputs $a_1, \dots, a_m, b_1, \dots, b_m$ for the synthetiser

Synthesisers from LWE?

- Hard to **tell apart** $(\mathbf{a}_i, \mathbf{b}_i = \langle \mathbf{a}_i, \mathbf{s} \rangle + e_i)$ from **random** $(\mathbf{a}, \mathbf{b})$
- By a **hybrid argument**, the following are **pseudorandom**

$$\mathbf{A}_i \in \mathbb{Z}_q^n, \mathbf{A}_i \cdot \mathbf{S}_1 + \mathbf{E}_{1,1} \in \mathbb{Z}_q^{n \times n}, \mathbf{A}_i \cdot \mathbf{S}_2 + \mathbf{E}_{2,1} \in \mathbb{Z}_q^{n \times n}, \dots$$

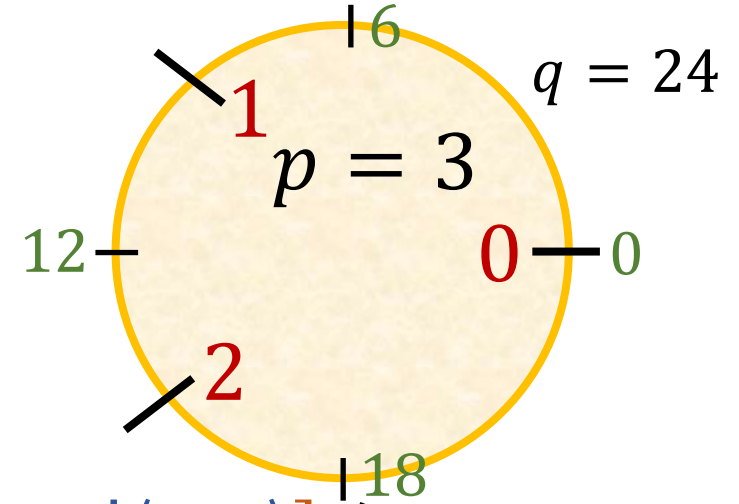
- This suggests the following synthesiser from LWE

	$\mathbf{S}_1$	$\mathbf{S}_2$	...
$\mathbf{A}_1$	$\mathbf{A}_1 \cdot \mathbf{S}_1 + \mathbf{E}_{1,1}$	$\mathbf{A}_1 \cdot \mathbf{S}_2 + \mathbf{E}_{1,2}$	
$\mathbf{A}_2$	$\mathbf{A}_2 \cdot \mathbf{S}_1 + \mathbf{E}_{2,1}$	$\mathbf{A}_2 \cdot \mathbf{S}_2 + \mathbf{E}_{2,2}$	
...			

- But synthesisers must be **deterministic**!

Learning with Rounding

- Generate errors **deterministically**
 - Round $\mathbb{Z}_q$ to a **sparse** subset $\mathbb{Z}_p$
 - For $p < q$, let $\lfloor x \rfloor_p = \lfloor (p/q) \cdot x \rfloor \bmod p$
- **Learning with Rounding (LWR):** Tell apart $(\mathbf{a}, b = \lfloor \langle \mathbf{a}, \mathbf{s} \rangle \rfloor_p) \in \mathbb{Z}_q \times \mathbb{Z}_p$ from **random** $(\mathbf{a}, b)$
 - LWE **conceals** low-order bits by adding **small random error**
 - LWR just **discards** those bits instead
- **LWE $\leq$ LWR** for $q \geq p \cdot n^{\omega(1)}$ (seems 2^n -hard for $q \geq p \cdot \sqrt{n}$)
 - Proof idea: w.h.p. $(\mathbf{a}, \lfloor \langle \mathbf{a}, \mathbf{s} \rangle + e \rfloor_p) \approx (\mathbf{a}, \lfloor \langle \mathbf{a}, \mathbf{s} \rangle \rfloor_p)$ and $(\mathbf{a}, \lfloor U(\mathbb{Z}_q) \rfloor_p) \approx (\mathbf{a}, U(\mathbb{Z}_p))$ where $U(\mathbb{Z}_q)$ is **uniform** over $\mathbb{Z}_q$
 - Reduction with **improved parameters** in subsequent works



Synthetiser-based PRF from LWR

- Synthetiser: $\mathcal{S}: \mathbb{Z}_q^{n \times n} \times \mathbb{Z}_q^{n \times n} \rightarrow \mathbb{Z}_p^{n \times n}$ is $\mathcal{S}(\mathbf{A}, \mathbf{S}) = \lfloor \mathbf{A} \cdot \mathbf{S} \rfloor_p$
 - Note that the range $\mathbb{Z}_p$ is **slightly smaller** than the domain $\mathbb{Z}_q$
- Construction of PRF with domain $\{0,1\}^k$ for $k = 2^d$
 - **Tower** of power moduli $q_d > q_{d-1} > \dots > q_0$
 - The secret key is $2k$ matrices $\mathbf{S}_{i,b} \in \mathbb{Z}_{q_d}^{n \times n}$, for $i \in [k], b \in \{0,1\}$
 - Depth $d = \log k$ of LWR synthetisers

$$\left[\left[\left[\mathbf{S}_{1,x_1} \cdot \mathbf{S}_{2,x_2} \right]_{q_2} \cdot \left[\mathbf{S}_{3,x_3} \cdot \mathbf{S}_{4,x_4} \right]_{q_2} \right]_{q_1} \cdot \left[\left[\mathbf{S}_{5,x_5} \cdot \mathbf{S}_{6,x_6} \right]_{q_2} \cdot \left[\mathbf{S}_{7,x_7} \cdot \mathbf{S}_{8,x_8} \right]_{q_2} \right]_{q_1} \right]_{q_0}$$

- Each synthetiser is in NC^1 , and thus the PRF is in NC^2

Direct Construction

- Simple **direct** PRF construction from DDH [NR97,NRR00]:

$$F_{g,s_1,\dots,s_k}(x_1, \dots, x_k) = g^{\prod_i s_i^{x_i}}$$

- This can be implemented in $TC^0 \subseteq NC^0$ (albeit with **huge** circuit)
- Direct construction from LWE
 - Public moduli $q > p$
 - The secret key is **uniform** A and **short** $s_1, \dots, s_k$ over $\mathbb{Z}_q$
 - The PRF evaluates a **rounded subset-product** function

$$F_{A,s_1,\dots,s_k}(x_1, \dots, x_k) = \left\lfloor A \cdot \prod_i s_i^{x_i} \right\rfloor_p$$

Proof Sketch

- Similar to the $\text{LWE} \leq \text{LWR}$ proof
- Thought experiment: answer queries with

$$\begin{aligned}\tilde{F}_{A, S_1, \dots, S_k}(x_1, \dots, x_k) &= \left[(A \cdot S_1^{x_1} + x_1 \cdot E) \cdot S_2^{x_2} \cdot \dots \cdot S_k^{x_k} \right]_p \\ &= \left[A \cdot \prod_{i=1}^k S_i^{x_i} + x_1 \cdot E \cdot \prod_{i=2}^k S_i^{x_i} \right]_p\end{aligned}$$

- W.h.p. $\tilde{F}(x) = F(x)$ due to **small error** and **rounding**
- Using LWE replace $(A, A \cdot S_1 + E)$ with uniform (A_0, A_1)
 - New function $F(x) = \left[A_{x_1} \cdot S_2^{x_2} \cdot \dots \cdot S_k^{x_k} \right]_p$
 - Repeat for $S_2, \dots, S_k$ to get $F' \dots'(x) = \left[A_x \right]_p = U(x)$

Crystals-Kyber

Regev PKE

- **Key Generation:** $pk = (A, b)$ and $sk = s$, where $b^t = s^t \cdot A + e^t$ and $s \in \mathbb{Z}_q^n, A \in \mathbb{Z}_q^{n \times m}$
- **Encryption:** The encryption of x w.r.t. pk is made of two parts
 - Ciphertext preamble $c_0 = A \cdot r$ for random $r \in \{0,1\}^m$
 - Ciphertext payload $c_1 = b^t \cdot r + x \cdot q/2$
 - Bob outputs $c_1 - s^t \cdot c_0 \approx x \cdot q/2$
- **Security:** Using LWE, we can switch (A, b) with (A, b) for **uniformly random b^t**
 - By the **leftover hash lemma**, we can finally replace c_0 with uniformly random c_0 , so that c_1 hides x **information theoretically**

Dual Regev

- **Key Generation:** $pk = (A, u)$ and $sk = r$, where $u = A \cdot r$ and $r \in \{0,1\}^m$, $A \in \mathbb{Z}_q^{n \times m}$
- **Encryption:** The encryption of x w.r.t. pk is made of two parts
 - Ciphertext preamble $c_0 = b^t = s^t \cdot A + e^t$ for random $s \in \mathbb{Z}_q^n$
 - Ciphertext payload $c_1 = s^t \cdot u + e' + x \cdot q/2$
 - Bob outputs $c_1 - c_0 \cdot r \approx x \cdot q/2$
- **Security:** By the leftover hash lemma, we can switch u with **uniformly random** u
 - By LWE, we can switch (c_0, c_1) with **uniformly random** (c_0, c_1)

Primal versus Dual

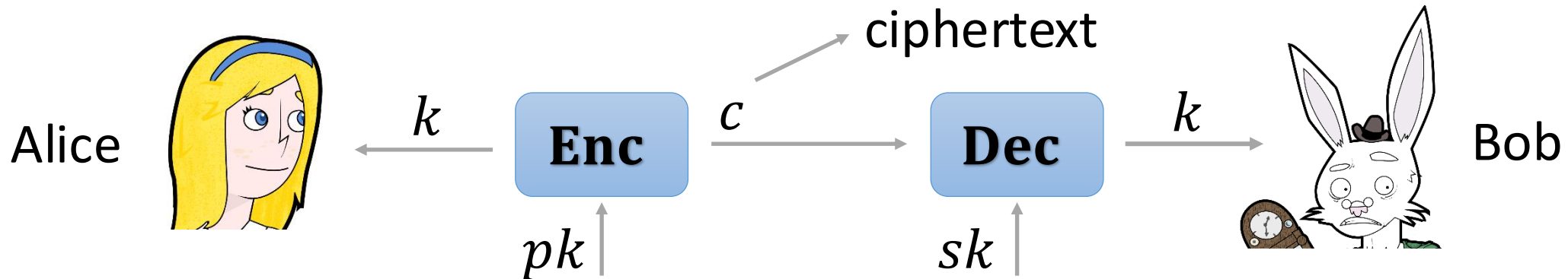
- Public key
 - Primal: pk is **pseudorandom** with **unique** sk
 - Dual: pk is **statistically random** with **many possible** sk
- Ciphertext
 - Primal: A fresh LWE sample with **many possible** coins
 - Dual: Multiple LWE samples with **unique** coins
- Security
 - Primal: Encrypting with **uniform** pk induces **random** ciphertext
 - Dual: By LWE can switch the ciphertext to **random**
- Efficiency: The matrix A can be **shared** by different users

Most Efficient

- **Key Generation**: $pk = (A, u)$ and $sk = s$, where $u^t = s^t \cdot A + e^t$ and $s \in \chi^n, A \in \mathbb{Z}_q^{n \times n}$
- **Encryption**: The encryption of x w.r.t. pk is made of two parts
 - Ciphertext preamble $c_0 = A \cdot r + e'$ for $r \in \chi^n$
 - Ciphertext payload $c_1 = u^t \cdot r + e' + x \cdot q/2$
 - Bob outputs $c_1 - s^t \cdot c_0 \approx x \cdot q/2$
- **Security**: Using LWE, we can switch (A, u) with (A, u) for **uniformly random u**
 - This requires LWE with secrets from the **error distribution**
 - Next, we can replace (c_0, c_1) with **uniformly random (c_0, c_1)**

Fujisaki-Okamoto Transform

- The **FO transform** turns **passively** (**IND-CPA**) secure PKE schemes into **actively** (**IND-CCA**) secure ones
 - The transformation requires two **hash functions** (random oracles)
 - The obtained scheme is better understood as a **key encapsulation mechanism** (KEM)

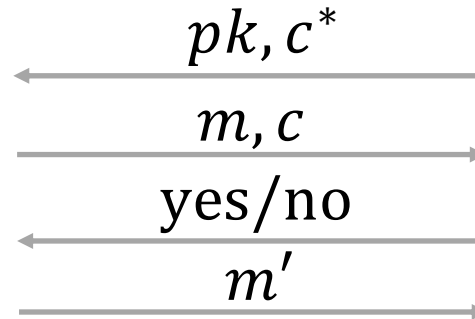


- We can combine a **KEM** with an **SKE** scheme to get a **PKE** scheme

One-Wayness of PKE



Eve



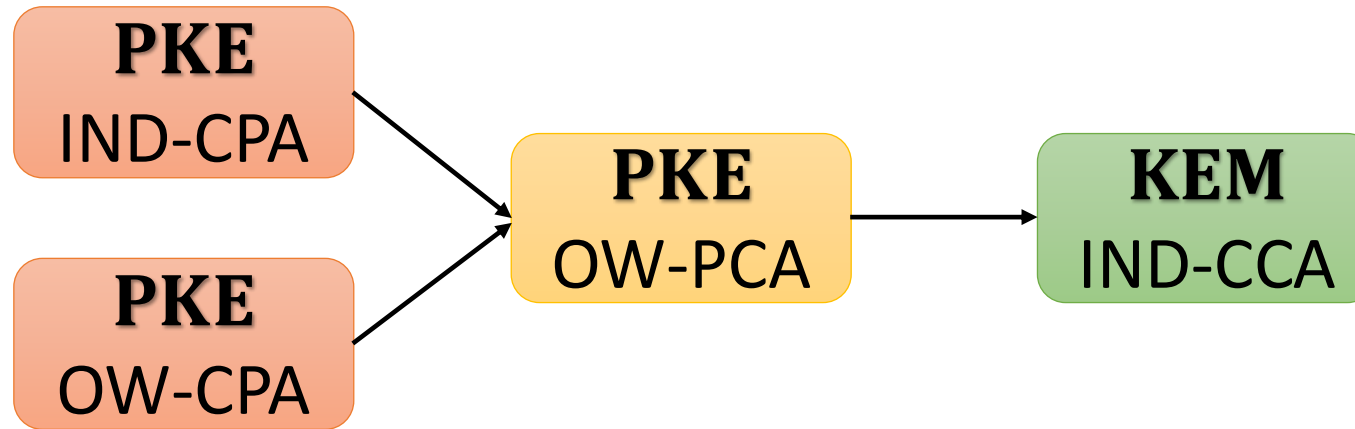
Challenger



pk, sk
 $m^* \leftarrow \mathcal{M}$
 $c^* \leftarrow \mathbf{Enc}(pk, m^*)$

- **OW-CPA**: PKE makes it **hard to guess** the message
 - The message is **uniformly random** and **unknown** to the attacker
- **OW-PCA**: As before but now the attacker can query a **plaintext-checking oracle** which allows to check if $\mathbf{Dec}(sk, c) = m$

Modularization of the FO Transform



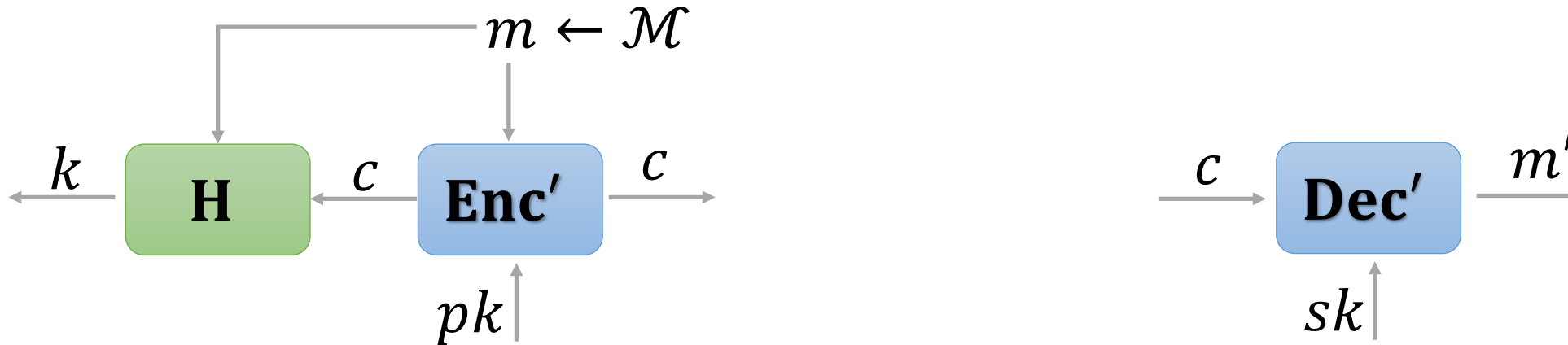
- We can view FO as the **concatenation** of **two transforms** $U \circ T$
 - The first transformation takes care of **derandomization** and allows to go from **IND-CPA** to **OW-PCA**
 - The second transformation takes care of **hashing** and allows to go from **OW-PCA** to **IND-CCA**

Transformation T: From IND-CPA to OW-PCA



- Encryption becomes **deterministic** (the **randomness** is $\mathbf{G}(m)$)
- Decryption **re-encrypts** m' using randomness $\mathbf{G}(m')$ and outputs m' if and only if it obtains c
- **Theorem:** Assuming $(\mathbf{Enc}, \mathbf{Dec})$ is **IND-CPA** (**OW-CPA**), $(\mathbf{Enc}', \mathbf{Dec}')$ is **OW-PCA**

Transformation U: From OW-PCA to IND-CCA



- Encapsulation outputs $k = \mathbf{H}(c, m)$ and c
- Decapsulation obtains $m' = \mathbf{Dec}(sk, c)$ and outputs m'
 - Here, m' could be $\perp$ (**explicit rejection**)
- **Theorem:** Assuming $(\mathbf{Enc}', \mathbf{Dec}')$ is **OW-PCA**, $(\mathbf{Encaps}, \mathbf{Decaps})$ is **IND-CCA**

Falcon

Lattice Trapdoors

- Recall: Lattice-based **one-way functions**

$$f_A(\mathbf{x}) = \mathbf{A} \cdot \mathbf{x} \bmod q \in \mathbb{Z}_q^n$$

(short $\mathbf{x}$, surjective)

$$f_A(\mathbf{s}, \mathbf{e}) = \mathbf{s}^t \cdot \mathbf{A} + \mathbf{e}^t \bmod q \in \mathbb{Z}_q^m$$

(short $\mathbf{e}$, injective)

- Task: **Invert** f_A
 - Find the **unique** $\mathbf{s}$ (or $\mathbf{e}$) such that $f_A(\mathbf{s}, \mathbf{e}) = \mathbf{s}^t \cdot \mathbf{A} + \mathbf{e}^t \bmod q$
 - Given $\mathbf{u} = f_A(\mathbf{x}') = \mathbf{A} \cdot \mathbf{x}' \bmod q$, **sample random** $\mathbf{x} \leftarrow f_A^{-1}(\mathbf{u})$ with probability proportional to $\exp(-\|\mathbf{x}\|^2/s^2)$
- How? Via a **strong trapdoor** for $\mathbf{A}$ (a **short basis** of $\mathcal{L}^\perp(\mathbf{A})$)
 - Deeply studied question

A Different Kind of Trapdoor

- Drawbacks of previous solutions
 - Generating A with short basis is **complex** and **slow**
 - Inversion algorithms trade-off **quality** (i.e., length of basis vectors which depends on the Gaussian std parameter s) for **efficiency**
- Alternative: The trapdoor is **not a basis**
 - But just **as powerful**
 - **Simpler** and **faster**
- Overview of method
 - Start with **fixed, public**, lattice defined by **gadget matrix** G which admits very **fast**, and **parallel**, algorithms for f_G^{-1}
 - **Randomize** G into A via nice **unimodular** transform (the trapdoor)
 - **Reduce** f_A^{-1} to f_G^{-1} plus some pre/post-processing

Step 1: The Gadget Matrix

- Let $q = 2^k$ and take $\mathbf{g} = [1 \quad 2 \quad \dots \quad 2^{k-1}] \in \mathbb{Z}_q^{1 \times k}$
- To invert $f_{\mathbf{g}}: \mathbb{Z}_q \times \mathbb{Z}^k \rightarrow \mathbb{Z}_q^k$

$$f_{\mathbf{g}}(s, \mathbf{e}) = s \cdot \mathbf{g} + \mathbf{e} = [s + e_0 \quad 2s + e_1 \quad \dots \quad 2^{k-1}s + e_{k-1}] \bmod q$$

- Get lsb of s from $2^{k-1}s + e_{k-1}$, then repeat for the next bits of s
- Works when $e_{k-1} \in [-q/4, q/4)$
- To sample **Gaussian preimage** for $u = f_{\mathbf{g}}(\mathbf{x}) = \langle \mathbf{g}, \mathbf{x} \rangle$
 - For $i \in [0, k-1]$, choose $x_i \leftarrow (2\mathbb{Z} + u)$ and let $u \leftarrow (u - x_i)/2 \in \mathbb{Z}$
 - E.g., $k = 2$: $x_0 \leftarrow (2z_0 + u)$, $u \leftarrow (u - 2z_0 - u)/2 = -z_0$, $x_1 \leftarrow (2z_1 - z_0)$,
 $\langle \mathbf{g}, \mathbf{x} \rangle = 2z_0 + u + 2(2z_1 - z_0) = u + 4z_1 = u \bmod 4$

Step 1: The Gadget Matrix G

- Alternative view: The **lattice** $\mathcal{L}^\perp(\mathbf{g})$ has **basis**

$$\mathbf{S} = \begin{bmatrix} 2 & & & & \\ -1 & 2 & & & \\ & -1 & \ddots & & \\ & & \ddots & 2 & \\ & & & -1 & 2 \end{bmatrix} \in \mathbb{Z}^{k \times k}, \text{ with } \tilde{\mathbf{S}} = 2 \cdot \mathbf{I}_k$$

- The above inversion algorithms are special cases of the randomized **nearest-plan algorithm**
- Define $\mathbf{G} = \mathbf{I}_n \otimes \mathbf{g} \in \mathbb{Z}^{n \times nk}$ (where $\otimes$ is the **tensor** product)
 - Computing $f_{\mathbf{G}}^{-1}$ reduces to n **parallel calls** to $f_{\mathbf{g}}^{-1}$
 - Also applies to $\mathbf{H} \cdot \mathbf{G}$, for any **invertible** $\mathbf{H} \in \mathbb{Z}_q^{n \times n}$

Step 2: Randomize G

- Define **semi-random** $[\bar{A}|G]$ for **uniform** $\bar{A} \in \mathbb{Z}_q^{n \times \bar{m}}$
 - It can be seen that inverting $f_{[\bar{A}|G]}^{-1}$ **reduces** to inverting f_G^{-1} [CHKP10]
- Choose a **short Gaussian** $R \in \mathbb{Z}^{\bar{m} \times n \log q}$ and let

$$A = [\bar{A}|G] \cdot \begin{bmatrix} I & R \\ & I \end{bmatrix} = [\bar{A}|G - \bar{A}R]$$

- A is **uniform** because, by the **leftover hash lemma**, $[\bar{A}|\bar{A}R]$ is **statistically close** to uniform when $\bar{m} \approx n \log q$
- Alternatively, $[I|\bar{A}] - \bar{A} \cdot R_1 + R_2$ is **pseudorandom** under the LWE assumption (in normal form)

A New Trapdoor Notion

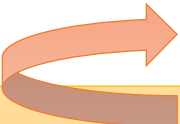
- We constructed $A = [\bar{A}|G - \bar{A}R]$
- Say that R is a **trapdoor** for A with **tag** $H \in \mathbb{Z}_q^{n \times n}$ (invertible) if

$$A \cdot \begin{bmatrix} R \\ I \end{bmatrix} = H \cdot G$$

- The **quality** of R is $s_1(R) = \max_{u: \|u\|=1} \|R \cdot u\|$
- **Fact:** $s_1(R) \approx (\sqrt{\text{rows}} + \sqrt{\text{cols}}) \cdot r$ for Gaussian entries w/ std dev r
- Also R is a trapdoor for $A - [0|H' \cdot G]$ with tag $H - H'$
- Relating new and old trapdoors
 - Given basis S for $\mathcal{L}^\perp(G)$ and trapdoor R for A , one can **efficiently** construct **basis** S_A for $\mathcal{L}^\perp(G)$ where $\|S_A\| \leq (s_1(R) + 1) \cdot \|S\|$

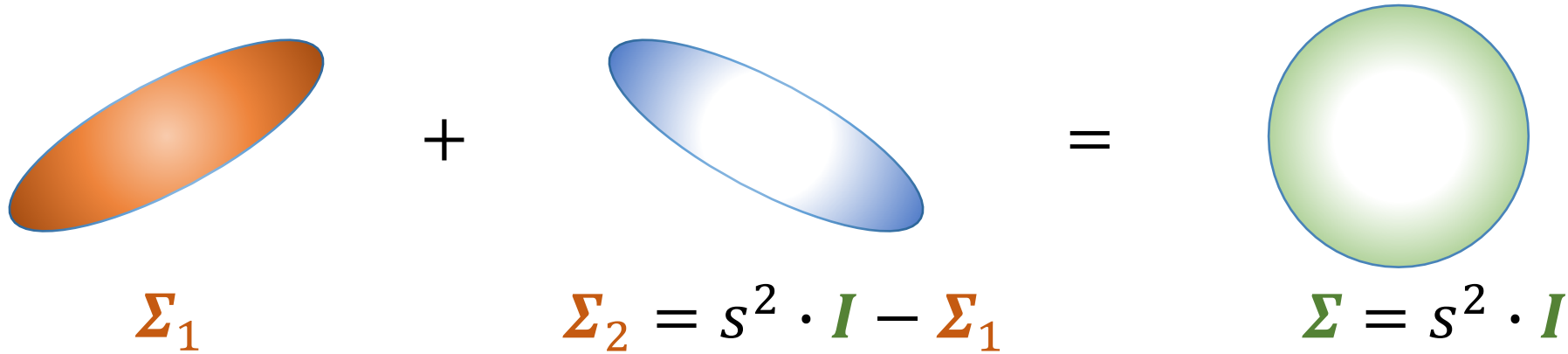
Step 3: Reduce f_A^{-1} to f_G^{-1}

- Let R be a **trapdoor** for A with **tag** $H = I$: $A \cdot \begin{bmatrix} R \\ I \end{bmatrix} = G$
- Inverting LWE
 - Given $b^t = s^t \cdot A + e^t$, recover s from $b^t \cdot \begin{bmatrix} R \\ I \end{bmatrix} = s^t \cdot G + e^t \cdot \begin{bmatrix} R \\ I \end{bmatrix}$
 - Works if **each entry** of $e^t \cdot \begin{bmatrix} R \\ I \end{bmatrix} \in [-q/4, q/4)$
- Inverting SIS
 - Given u , sample $z \leftarrow f_G^{-1}(u)$ and output $x = \begin{bmatrix} R \\ I \end{bmatrix} \cdot z \in f_A^{-1}(u)$
 - Indeed, $A \cdot x = G \cdot z = u$

 **Leaks about R !**

$$\Sigma = \mathbb{E}_x[x \cdot x^t] = \mathbb{E}_z[R \cdot z \cdot z^t \cdot R^t] \approx R \cdot R^t$$

Step 3: Perturbation Method



- To **fix** the covariance

- Generate **perturbation** vector p with covariance $s^2 \cdot I - R \cdot R^t$
- Sample **spherical** z such that $G \cdot z = u - A \cdot p$
- Output $x = p + \begin{bmatrix} R \\ I \end{bmatrix} \cdot z$

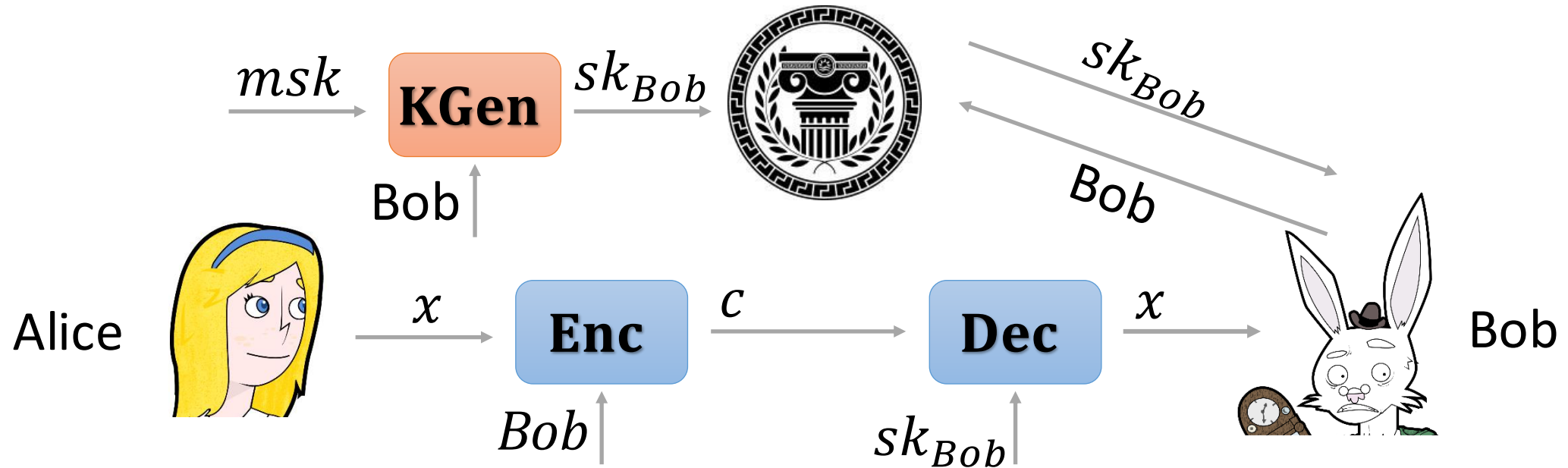
$$A \cdot x = A \cdot p + A \cdot \begin{bmatrix} R \\ I \end{bmatrix} \cdot z = A \cdot p + G \cdot z = u$$

Falcon: Digital Signatures from SIS

- Generate **uniform** $vk = A$ with **trapdoor** $sk = T$
- To sign μ , use T to **sample** $\sigma = x \in \mathbb{Z}^m$ such that $A \cdot x = H(\mu)$, where H is a **public** hash function
 - Recall that x is drawn from a **Gaussian distribution**, which **reveals nothing** about the trapdoor T
- To verify $(\mu, \sigma = x)$ under $vk = A$ simply check $A \cdot x = H(\mu)$ and that x is **sufficiently short**
- Security: **Forging** a signature for a new message μ^* requires finding a **short** x^* such that $A \cdot x^* = H(\mu^*)$
 - This is **equivalent** to solving the SIS problem
 - Signatures queries **do not help** because they **reveal nothing** about the trapdoor T

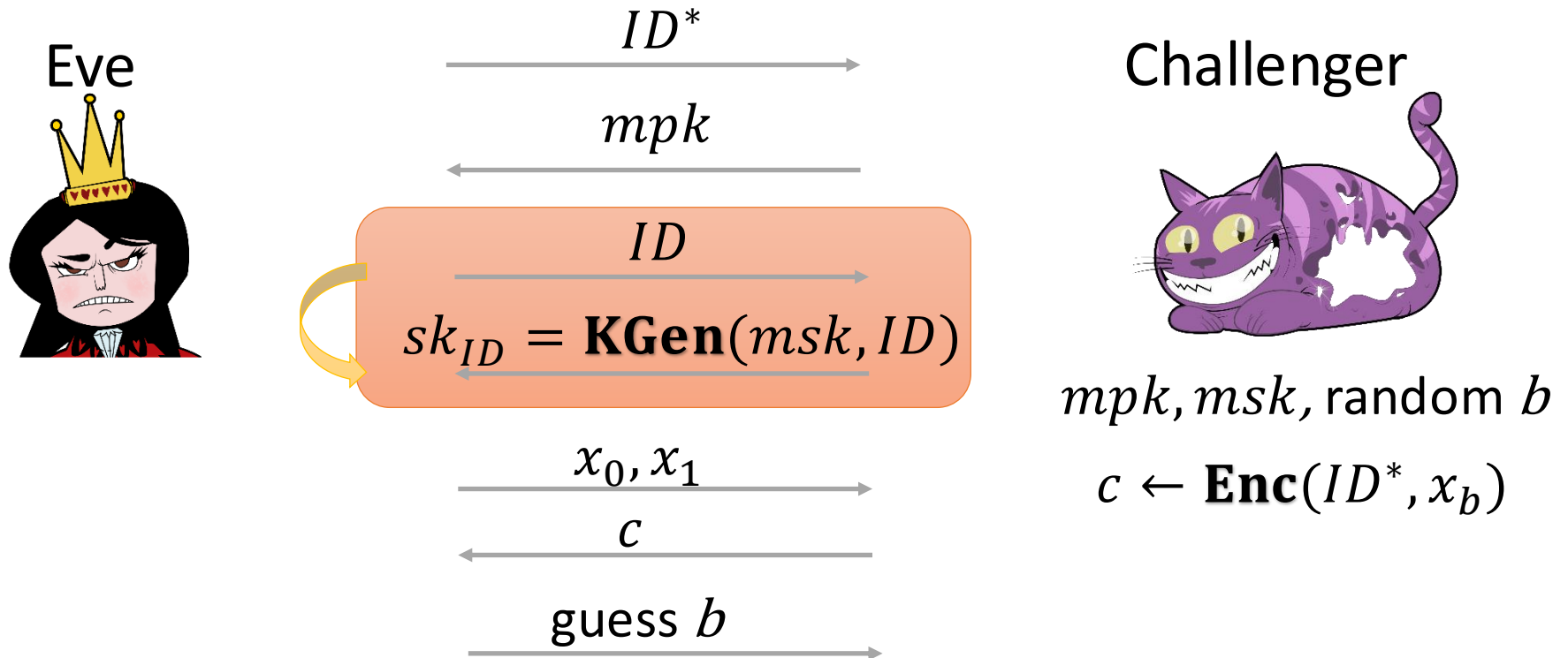
Other Applications of Lattice Trapdoors

Identity-Based Encryption



- **Postulated** by Shamir in 1984
 - Avoids the need of **certificates**
 - Introduces the so-called **key escrow** problem
- First **realization** by Boneh and Franklin in 2001 using **pairings**

Selective Security of IBE



- Every **selectively** secure IBE is also **fully** secure with an **exponential** loss in the parameters
 - Also, general **transformations** are known

Warm-up Construction

- **Public parameters:** $mpk = (A_0, A_1^0, A_1^1, A_2^0, A_2^1, u)$
 - Assume, for simplicity, $|ID| = 2$
- **Master secret key:** Trapdoor for A_0
 - Secret key for identity $ID = 01$: **Short vector** s s.t. $F_{01} \cdot s = u \bmod q$, where $F_{01} = [A_0 | A_1^0 | A_2^1]$
 - Note: A trapdoor for A_0 **implies** a trapdoor for F_{01}
- **Encryption: Dual** Regev encryption of x w.r.t. matrix F_{01}
 - The ciphertext is $c_0^t = r^t \cdot F_{01} + e^t$ and $c_1 = r^t \cdot u + e' + x \cdot q/2$
 - Bob outputs $c_1 - c_0^t \cdot s \approx x \cdot q/2$

Simulation

- Assume the **challenge** identity is $ID^* = 11$
 - The reduction **can't know** the secret key for ID^*
- Choose A_0, A_1^1, A_2^1 uniformly at **random**, but sample A_1^0, A_2^0 with the corresponding **trapdoors**
- The reduction can derive trapdoors for $F_{00} = [A_0 | A_1^0 | A_2^0]$, $F_{01} = [A_0 | A_1^0 | A_2^1]$, and $F_{10} = [A_0 | A_1^1 | A_2^0]$ but not for $F_{11} = [A_0 | A_1^1 | A_2^1]$
 - This allows the reduction to **simulate** key extraction queries while **embedding** the LWE challenge in the simulation

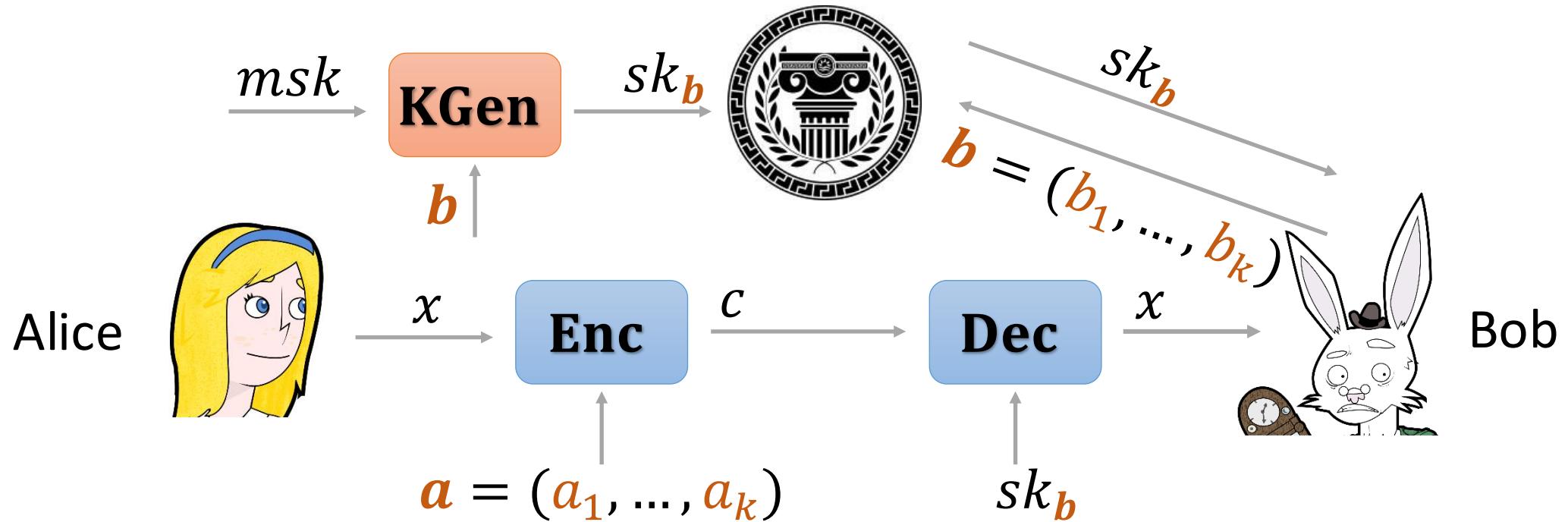
A More Efficient Construction

- **Public parameters:** $mpk = (A_0, A_1, G, u)$
- **Master secret key:** Trapdoor for A_0
 - Secret key for identity ID : **Short vector** s s.t. $F_{ID} \cdot s = u \mod q$, where $F_{ID} = [A_0 | A_1 + G]$
 - As before, a trapdoor for A_0 **implies** a trapdoor for F_{ID}
- **Encryption: Dual** Regev encryption of x w.r.t. matrix F_{ID}
 - The ciphertext is $c_0^t = r^t \cdot F_{ID} + e^t$ and $c_1 = r^t \cdot u + e' + x \cdot q/2$
 - Bob outputs $c_1 - c_0^t \cdot s = r^t \cdot u + e' + x \cdot q/2 - r^t \cdot F_{ID} \cdot s + e^t \cdot s = r^t \cdot u + e' + x \cdot \frac{q}{2}$

Simulation Revisited

- Assume the **challenge** identity is ID^*
 - The reduction **can't know** the secret key for ID^*
- The reduction does **not** know a trapdoor for A_0 , but it knows a trapdoor for the gadget matrix G
- Let $A_1 = [A_0 \cdot R - ID^* \cdot G]$, where R is **random** and **low-norm**
 - This is **indistinguishable** from the real A_1
- Note that $F_{ID} = [A_0 | A_0 \cdot R + (ID - ID^*) \cdot G]$
 - Again, we can **derive** a trapdoor for F_{ID} given a trapdoor for G
 - This allows to **simulate** key extraction queries for all $ID \neq ID^*$
 - The LWE challenge can be **embedded** as before

Inner-product Encryption

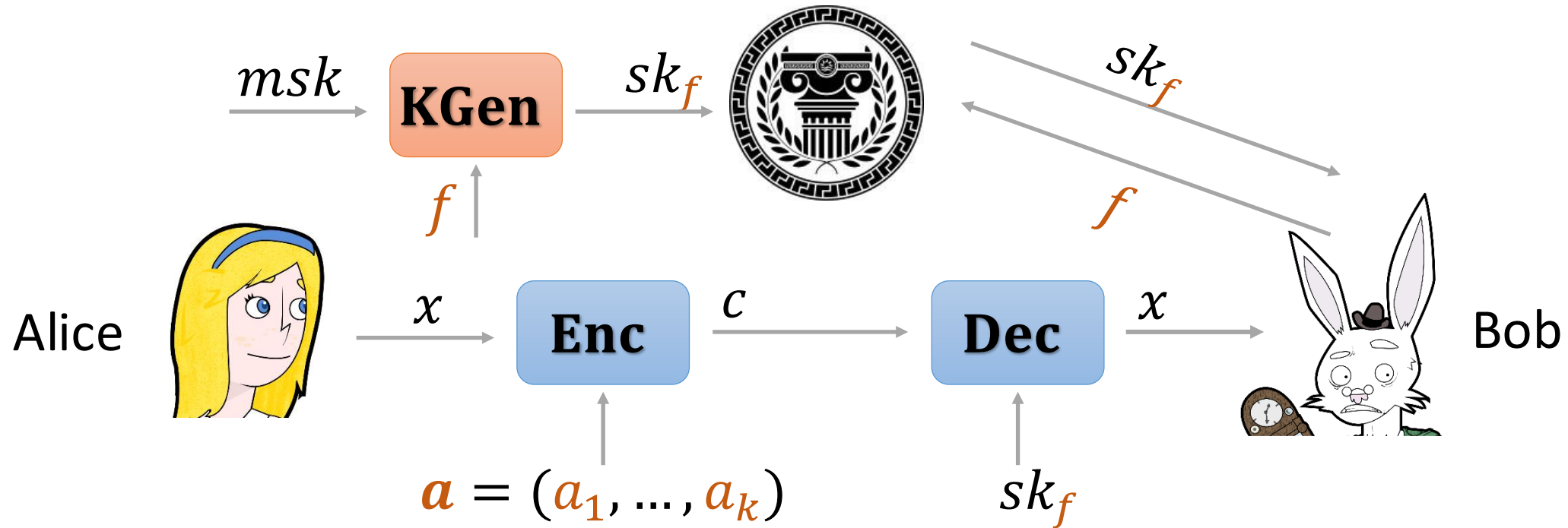


- Decryption reveals x **if and only if** $\langle a, b \rangle = 0$
 - Here, we can also be interested in **attributes privacy**
- Can be used to obtain **predicate encryption** for polynomial evaluation, CNFs/DNFs of bounded degree, and **fuzzy** IBE

Generalizing to Inner Products

- **Public parameters:** $mpk = (A, A_1, \dots, A_k, G, u)$
- **Master secret key:** Trapdoor for A
 - Secret key for b : **Short vector** s_b s.t. $F_b \cdot s_b = u \bmod q$, where $F_b = [A \mid \sum_i b_i \cdot A_i]$
- **Encryption: Dual** Regev encryption of x w.r.t. matrix A
 - The ciphertext is $c_0^t = r^t \cdot A + e^t$, $c' = r^t \cdot u + e' + x \cdot q/2$, and $c_i^t = r^t \cdot (A_i + a_i \cdot G) + e_i^t$ (so it indeed hides a)
 - Bob sets $c_b = \sum_i b_i \cdot c_i = r^t \cdot (\sum_i b_i \cdot A_i + \sum_i a_i \cdot b_i \cdot G) + \sum_i b_i \cdot e_i$ which equals $r^t \cdot \sum_i b_i \cdot A_i + \sum_i b_i \cdot e_i$
 - Hence, $[c_0 \mid c_b] \approx r^t \cdot [A \mid \sum_i b_i \cdot A_i]$ is a dual Regev ciphertext
 - Bob outputs $c' - c_0^t \cdot s_b - c_b^t \cdot s_b \approx x \cdot q/2$

Attribute-based Encryption



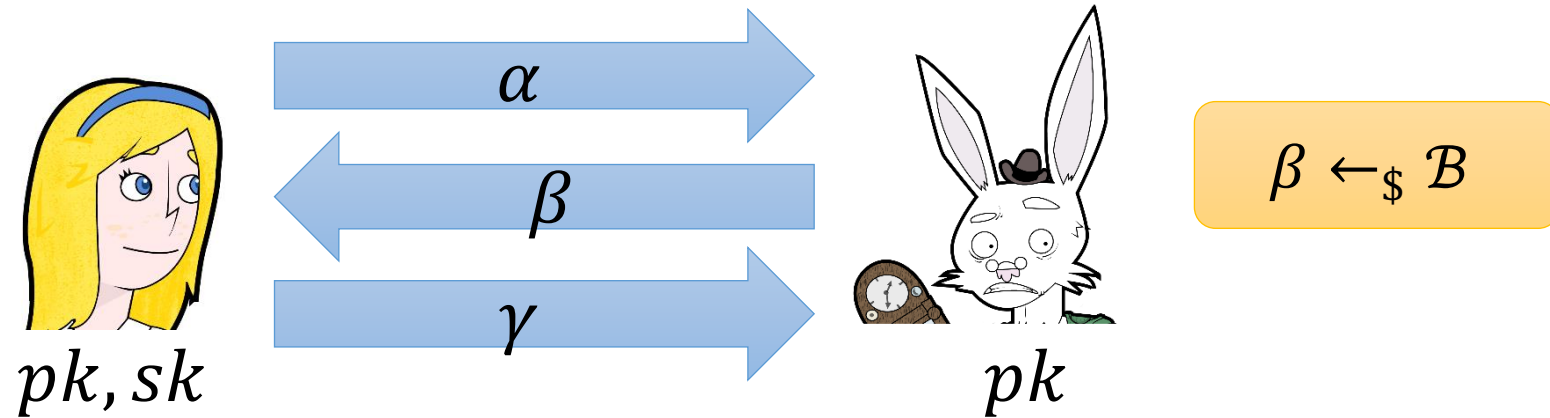
- Decryption reveals x **if and only if** $f(a) = 0$
 - Here, we are not interested in **attributes privacy**
- Plenty of applications for **privacy-preserving data mining** and in cryptography for **big data**

Handling Multiplications

- Let $\mathbf{c}_1^t = \mathbf{r}^t \cdot (\mathbf{A}_1 + a_1 \cdot \mathbf{G}) + \mathbf{e}_1^t$ and $\mathbf{c}_2^t = \mathbf{r}^t \cdot (\mathbf{A}_2 + a_2 \cdot \mathbf{G}) + \mathbf{e}_2^t$
- Want: $\mathbf{c}_{12}^t = \mathbf{r}^t \cdot (\mathbf{A}_{12} + a_1 \cdot a_2 \cdot \mathbf{G}) + \mathbf{e}_{12}^t$
 - Compute $(\mathbf{A}_1 + a_1 \cdot \mathbf{G}) \cdot \mathbf{G}^{-1}(-\mathbf{A}_2) = \mathbf{A}_1 \cdot \mathbf{G}^{-1}(-\mathbf{A}_2) - a_1 \cdot \mathbf{A}_2$
 - Compute $(\mathbf{A}_2 + a_2 \cdot \mathbf{G}) \cdot a_1 = a_1 \cdot \mathbf{A}_2 + a_1 \cdot a_2 \cdot \mathbf{G}$
 - The **difference** is $\mathbf{A}_{12} + a_1 \cdot a_2 \cdot \mathbf{G}$
- So, we let $\mathbf{c}_{12}^t = \mathbf{c}_1^t \cdot \mathbf{G}^{-1}(-\mathbf{A}_2) + \mathbf{c}_2^t \cdot a_1$
 - $\mathbf{G}^{-1}(-\mathbf{A}_2)$ and a_1 are **small** and **do not effect noise**
 - As usual, additionally let $\mathbf{c}_0^t = \mathbf{r}^t \cdot \mathbf{A} + \mathbf{e}^t$, $c' = \mathbf{r}^t \cdot \mathbf{u} + e' + x \cdot q/2$
 - If $a_1 \cdot a_2 = 0$, then $[\mathbf{c}_0 | \mathbf{c}_{12}] \approx \mathbf{r}^t \cdot [\mathbf{A} | \mathbf{A}_{12}]$
 - The secret key is a **short vector** $\mathbf{s}_{12}$ s.t. $[\mathbf{A} | \mathbf{A}_{12}] \cdot \mathbf{s}_{12} = \mathbf{u} \bmod q$
 - Bob outputs $c' - \mathbf{c}_0^t \cdot \mathbf{s}_{12} - \mathbf{c}_{12}^t \cdot \mathbf{s}_{12} \approx x \cdot q/2$

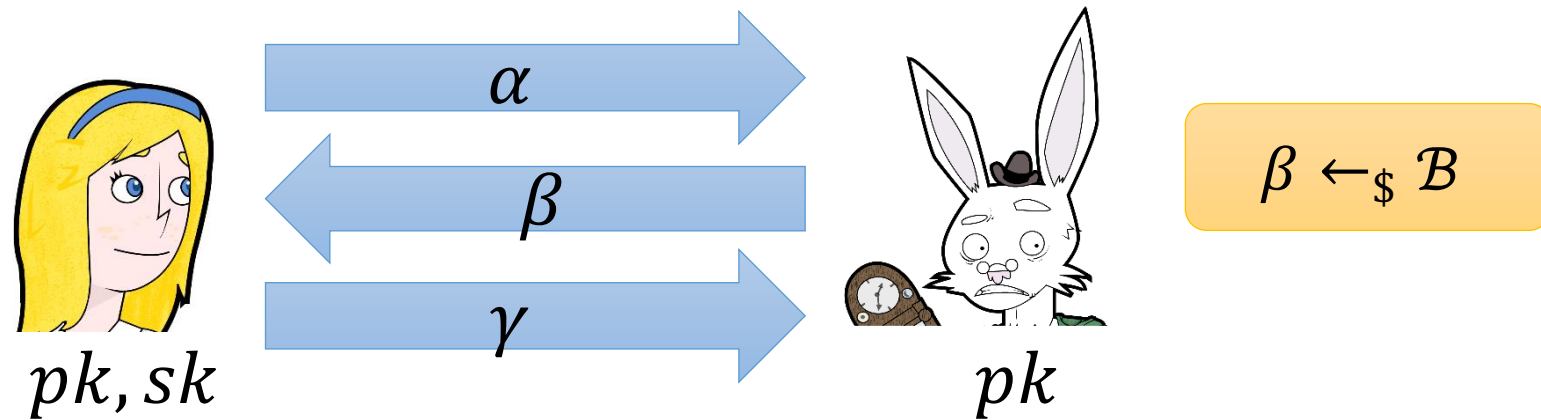
Crystals-Dilithium

Canonical Identification Schemes



- **Completeness:** The **honest** prover convinces the **honest** verifier (with all but a negligible probability)
- **Passive Security:** No (**efficient**) **malicious** prover knowing only pk can convince the **honest** verifier
 - Even in case the attacker knows many **accepting transcripts** corresponding to **honest** protocol executions

Sufficient Criteria for Passive Security



- One can show the following criteria are **sufficient** for achieving **passive security**:
 - **Special soundness**: Given any pk and two **accepting** transcripts (α, β, γ) and $(\alpha, \beta', \gamma')$ for pk with $\beta \neq \beta'$, there is a polynomial-time algorithm **outputting** sk
 - **HVZK**: **Honest** transcripts **reveal nothing** about the secret key sk

Proofs of Knowledge

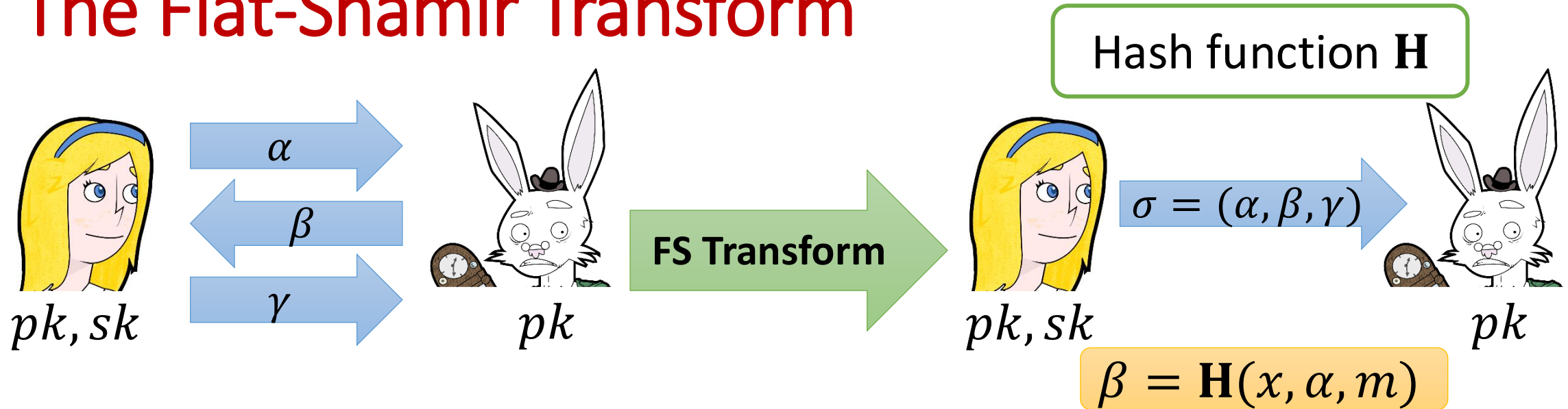
- The **special soundness** property implies that any successful prover must essentially **know the secret key**
- In fact, any such prover can be used to **extract** the secret key:
 - Run the prover upon input pk in order to obtain a transcript (α, β, γ)
 - **Rewind** the prover after it already sent α and forward it **another random challenge** β' , which yields a transcript $(\alpha, \beta', \gamma')$
 - As long as $\beta \neq \beta'$, **special soundness** allows us to obtain sk
- The above can be formalized, but the proof requires **some care**
 - Because the transcripts (α, β, γ) and $(\alpha, \beta', \gamma')$ are **correlated**

Honest-Verifier Zero-Knowledge

- How do we formalize that a transcript **reveals nothing** on sk ?
 - This is tricky: transcripts shall not reveal even **one bit** of sk
- Require that honest transcripts can be **efficiently simulated** given just pk (but not sk)
 - Whatever the verifier could compute via the protocol, he could have computed by **talking to himself** (i.e., by running the simulator)
- A canonical ID scheme is **perfect honest-verifier zero-knowledge** (HVZK) if $\exists$ PPT $\mathcal{S}$ such that:

$$(pk, sk, \mathcal{S}(pk)) \equiv (pk, sk, \langle \mathcal{P}(pk, sk), \mathcal{V}(pk) \rangle)$$

The Fiat-Shamir Transform



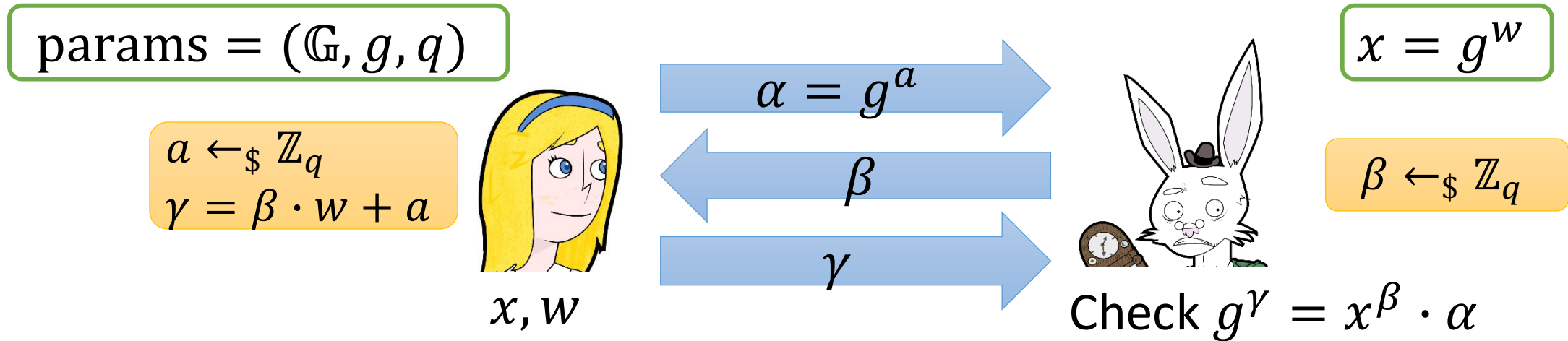
- Given a **canonical** ID scheme, we can derive a **signature scheme** as follows:
 - Alice obtains $\sigma = (\alpha, \beta, \gamma)$ from the **prover**, using the **secret key** sk and choosing $\beta = H(x, \alpha, m)$
 - Bob checks that (α, β, γ) is a **valid transcript**, with $\beta = H(x, \alpha, m)$
- Need to assume that the commitment α has high enough **min-entropy**

The Fiat-Shamir Transform

Theorem: If the ID scheme is **passively** secure, the signature derived via the **Fiat-Shamir** transform is **UF-CMA**

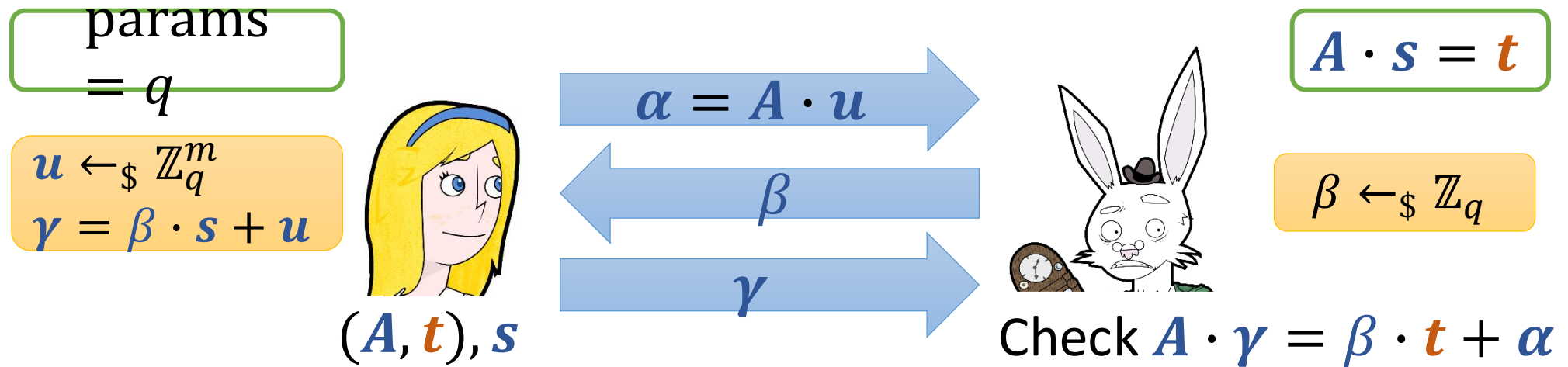
- **Remark:** The original proof requires to model **H** as an **ideal** hash function (**random oracle**)
 - It is **debatable** in the community what such a proof means in **practice**
- Can we prove security in the **plain model** (i.e., no random oracles)?
 - Many **impossibility** results for **general** ID schemes
 - **Possible** for **some** classes of ID schemes assuming so-called **correlation intractability**

Canonical ID Scheme from Discrete Log



- **HVZK:** Upon input $pk = x$, **simulator** $\mathcal{S}$ outputs (α, β, γ) such that $\alpha = g^\gamma / x^\beta$ and $\beta, \gamma \leftarrow_{\$} \mathbb{Z}_q$
- **Special soundness:** Assume we are given two accepting transcripts (α, β, γ) and $(\alpha, \beta', \gamma')$ for $pk = x$, with $\beta \neq \beta'$
 - This implies $g^{\gamma - \gamma'} = x^{\beta - \beta'}$
 - Thus, $w = (\gamma - \gamma') \cdot (\beta - \beta')^{-1}$ is the **discrete logarithm** of x

Let's Try the Same Idea using Lattices

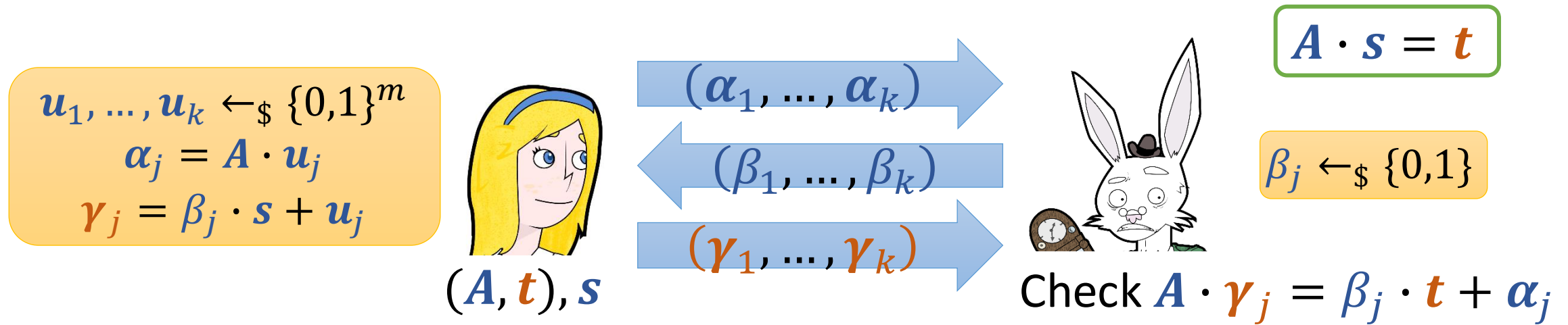


- **HVZK:** Upon input $pk = (A, t)$, **simulator** $\mathcal{S}$ outputs (α, β, γ) such that $\alpha = A \cdot \gamma - \beta \cdot t$ and $\beta \leftarrow_{\$} \mathbb{Z}_q, \gamma \leftarrow_{\$} \mathbb{Z}_q^m$
- **Special soundness:** Assume we are given two accepting transcripts (α, β, γ) and $(\alpha, \beta', \gamma')$ for $pk = (A, t)$, with $\beta \neq \beta'$
 - This implies $A \cdot (\gamma - \gamma') = (\beta - \beta') \cdot t$
 - Thus, $s = (\gamma - \gamma') \cdot (\beta - \beta')^{-1}$ is the **solution** for $A \cdot s = t$

Many Problems...

- The challenge space is **small**
 - $q \approx 2^{12}$ for **encryption**
 - $q \approx 2^{30}$ for **signatures**
 - $q \approx 2^{32}$ for **advanced applications**
- This means that a **successful prover** can just **guess** β
- The vector $\mathbf{s}$ we extract is **not guaranteed to be small**
 - Recall that **removing** the requirement of $\mathbf{s}$ being **small** makes the underlying lattice problems **trivial**
- **Solution:** Choose **small** $\mathbf{u}, \beta$ and **repeat** the protocol in **parallel**

Modified Protocol (Take 1)



- The verifier checks the above $\forall j = 1, \dots, k$ and that the coefficients of each γ_j are **small** (i.e., in $\{0,1,2\}$)
- **Special soundness:** Given $A \cdot \gamma_j = \beta_j \cdot t + \alpha_j$ and $A \cdot \gamma'_j = \beta'_j \cdot t + \alpha_j$ with $\beta_j \neq \beta'_j$, extract $s = (\gamma_j - \gamma'_j) \cdot (\beta_j - \beta'_j)^{-1}$
 - The elements of $\gamma_j - \gamma'_j$ are in $\{-2, -1, 0, 1, 2\}$, and $\beta_j - \beta'_j$ is in $\{-1, 1\}$, so s also lies in $\{-2, -1, 0, 1, 2\}$

Insecurity of the Protocol

- There are some **caveats**:
 - We **extracted** a **slightly bigger** secret
 - We need to **repeat** for $k = 128$ or $k = 256$ times
- Even worse, the protocol **does not** satisfy **HVZK**
 - Suppose that the challenge is $\beta = 1$

0 ? 1 ? 1 0 ? 0 ? ?

+

0 ? 1 ? 1 0 ? 0 ? ?

=

0 1 2 1 2 0 1 0 1 1

$\beta \cdot s = s$ has coefficients in $\{0,1\}$

u has coefficients in $\{0,1\}$

γ coefficients

Possible Fix?

- Maybe we can sample u from a **larger domain**?
 - Suppose that the challenge is $\beta = 1$

$$\begin{array}{cccccccccc} 0 & ? & ? & ? & 1 & ? & 0 & ? & ? & ? \\ + \\ 0 & ? & ? & ? & 5 & ? & 0 & ? & ? & ? \\ = \\ 0 & 4 & 2 & 3 & 6 & 5 & 0 & 2 & 4 & 1 \end{array}$$

$\beta \cdot s = s$ has coefficients in $\{0,1\}$

u has coefficients in $\{0,1,2,3,4,5\}$

γ coefficients

- Whenever a γ coefficient is 0 or 6 we know that s is 0 or 1, but the other coefficients are **hidden** (i.e., they could be **equally** 0 or 1)
- So, s **only** effects the probability that a γ coefficient is 0 or 6

Possible Fix?

- Maybe we can sample u from a **larger domain**?
 - Suppose that the challenge is $\beta = 1$

$$\begin{array}{cccccccccc} 0 & ? & ? & ? & 1 & ? & 0 & ? & ? & ? \\ + \\ 0 & ? & ? & ? & 5 & ? & 0 & ? & ? & ? \\ = \\ 0 & 4 & 2 & 3 & 6 & 5 & 0 & 2 & 4 & 1 \end{array}$$

$\beta \cdot s = s$ has coefficients in $\{0,1\}$

u has coefficients in $\{0,1,2,3,4,5\}$

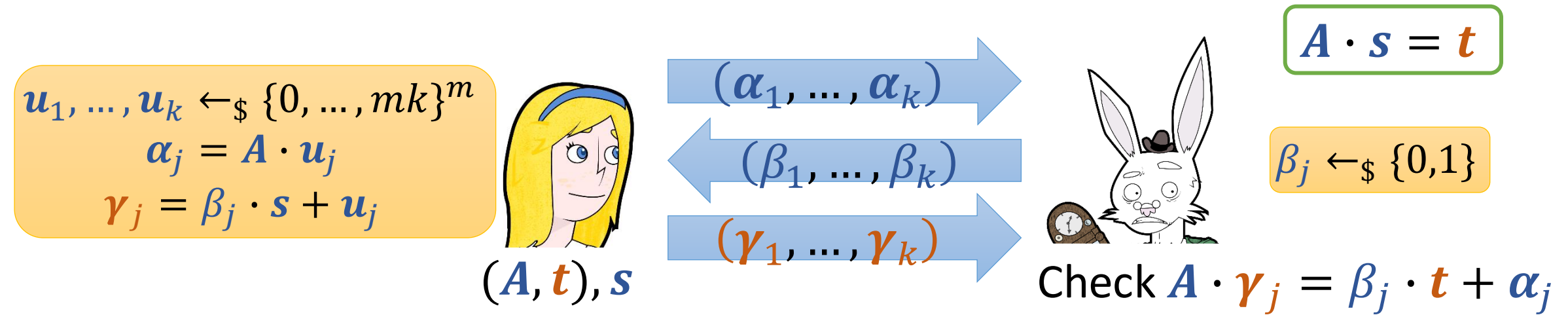
γ coefficients

- In other words, the coefficients 1,2,3,4,5 are **equally likely** to appear **regardless** of the **secret key**
- Natural idea: Send γ only when **all the coefficients** are **in this range**

In General...

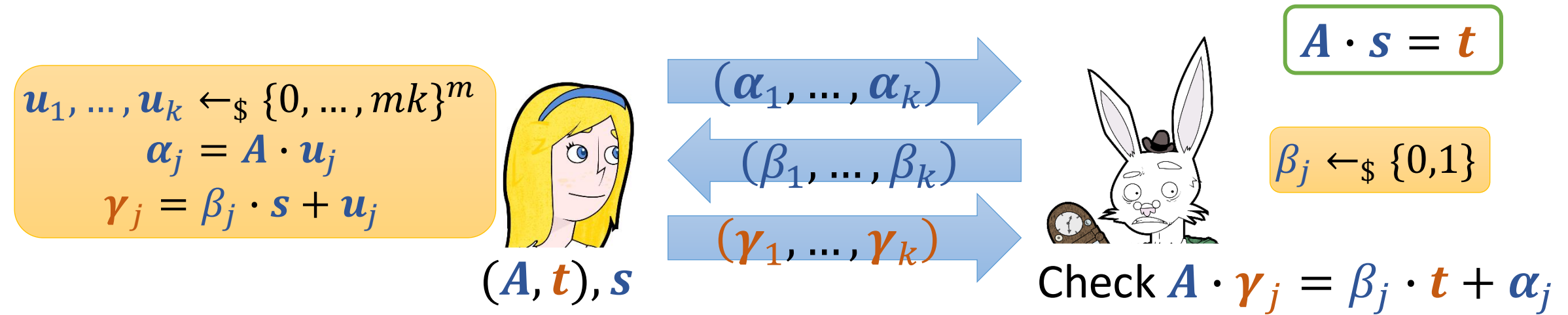
- Suppose $\mathbf{s}$ has coefficients in $\{0, 1, \dots, a\}$ and that $\mathbf{u}$ has coefficients in $\{0, 1, \dots, b - 1\}$
 - Here, $b > a$
- Then, for all $a \leq i < b$, we have $\mathbb{P}[\mathbf{s} + \mathbf{u} = \mathbf{i}] = 1/b$
 - Moreover, there are $b - a$ such i 's and thus $1 - a/b$ probability of keeping the value $\mathbf{s}$ **secret**
- The probability that a γ coefficient is in $\{1, \dots, b - 1\}$ is $1 - 1/b$
 - The probability that they **all are** is $(1 - 1/b)^m$
 - The probability that they **all are for all** $\gamma_1, \dots, \gamma_k$ is $(1 - 1/b)^{mk}$
 - By setting $b = mk$, we get $(1 - 1/b)^{mk} \approx 1/e$

Modified Protocol (Take 2)



- The prover checks whether **any** of the coefficients contained in γ_j is 0 or $mk + 1$
 - If it is, **abort** and **restart** the protocol
- The verifier checks the above $\forall j = 1, \dots, k$ and that the coefficients of each γ_j are **small** (i.e., in $\{0, \dots, mk\}$)

Modified Protocol (Take 2)



- **Special soundness**: Given $A \cdot \gamma_j = \beta_j \cdot t + \alpha_j$ and $A \cdot \gamma'_j = \beta'_j \cdot t + \alpha_j$ with $\beta_j \neq \beta'_j$, extract $s = (\gamma_j - \gamma'_j) \cdot (\beta_j - \beta'_j)^{-1}$
 - The elements of $\gamma_j - \gamma'_j$ are in $\{-mk, \dots, mk\}$, and $\beta_j - \beta'_j$ is in $\{-1, 1\}$, so s also lies in $\{-mk, \dots, mk\}$
- **HVZK**: Yes, as now γ_j **never depends** on s
 - **Caveat**: What is α_j in case of **abort**?

In Practice

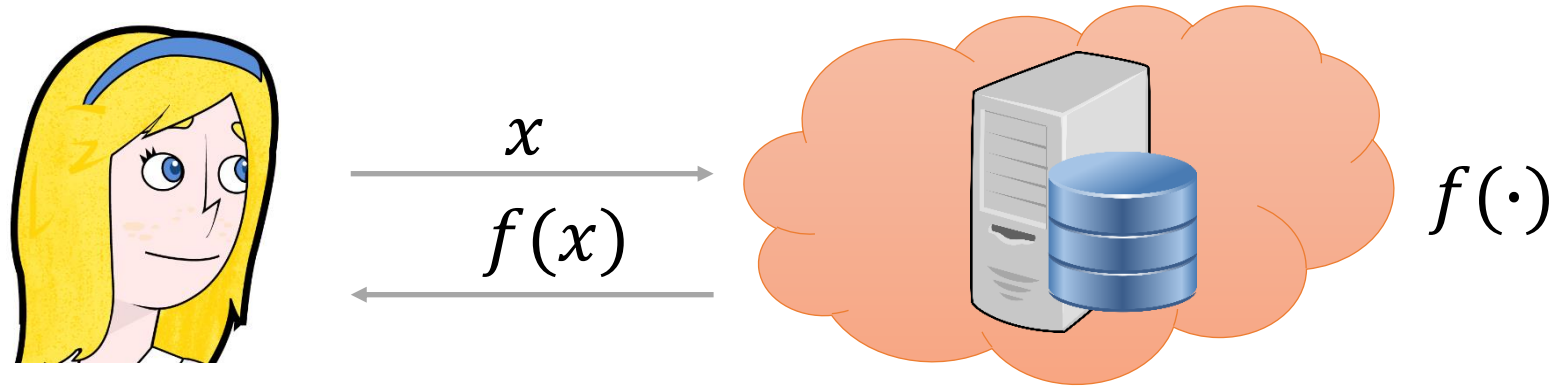
- The previous protocol still needs to be **repeated in parallel** $k = 128$ or 256 times
 - And this is the best one can get for **arbitrary** lattices
- However:
 - The proof size for **one equation** is roughly the same as the proof size for **many equations** (amortization with **logarithmic** growth)
 - Working with **polynomial rings** instead of $\mathbb{Z}_q$ allows for **one-shot approximate** proofs (i.e., the coefficients of $\mathbf{s}$ are **small**)
 - Using more **complex techniques**, one obtains **almost one-shot exact** proofs (i.e., the coefficients of $\mathbf{s}$ are in $\{0,1\}$)

Homomorphic Encryption

Computing over Encrypted Data

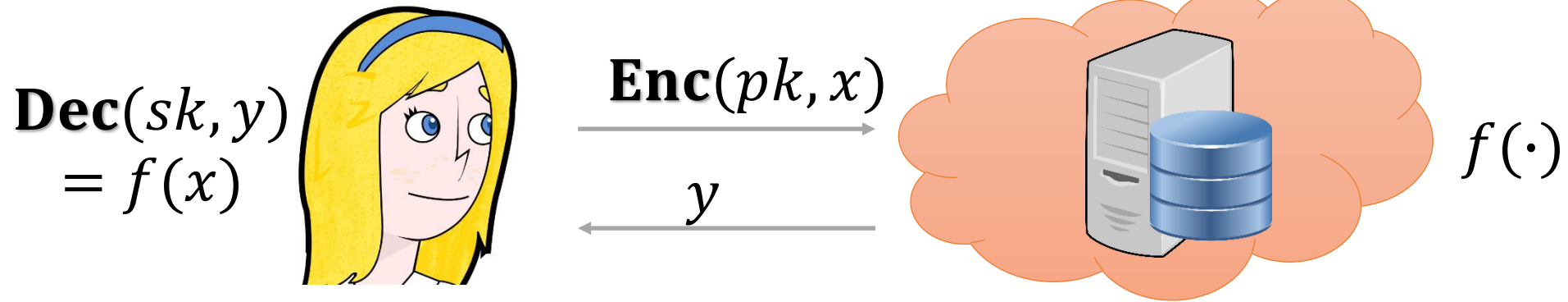
- Can we have a (public-key) encryption scheme which allows to run **computations** over **encrypted data**?
- Question dating back to the late 70s
 - Ron Rivest and "privacy homomorphisms"
- Partial solutions known
 - E.g., RSA and Elgamal enjoy limited forms of homomorphism
- First solution by Craig Gentry after 30 years
 - The "Swiss Army knife of cryptography"

Motivation: Outsourcing of Computation



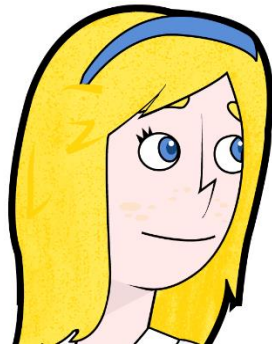
- Email, web search, navigation, social networking, ...
- What about **private** x ?

Outsourcing of Computation - Privately



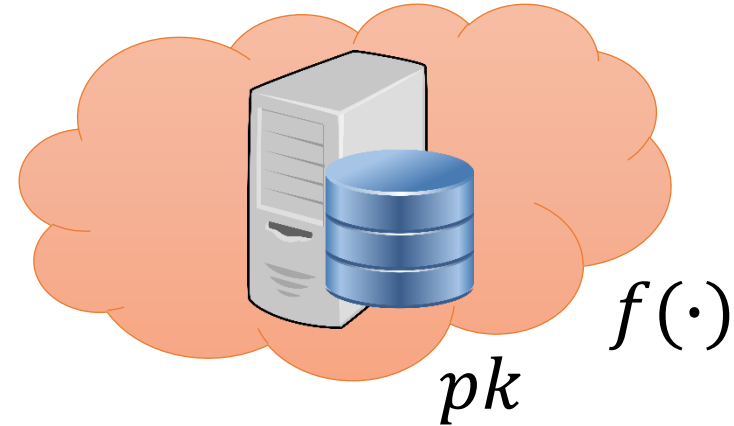
Wish: Homomorphic **evaluation** function:
Eval: $pk, f, \text{Enc}(pk, x) \rightarrow \text{Enc}(pk, f(x))$

Fully-Homomorphic Encryption (FHE)



pk, sk

$$\begin{aligned} c &= \mathbf{Enc}(pk, x) \\ y &= \mathbf{Eval}(pk, f, c) \end{aligned}$$



Correctness:

$$\mathbf{Dec}(sk, y) = f(x)$$

Privacy:

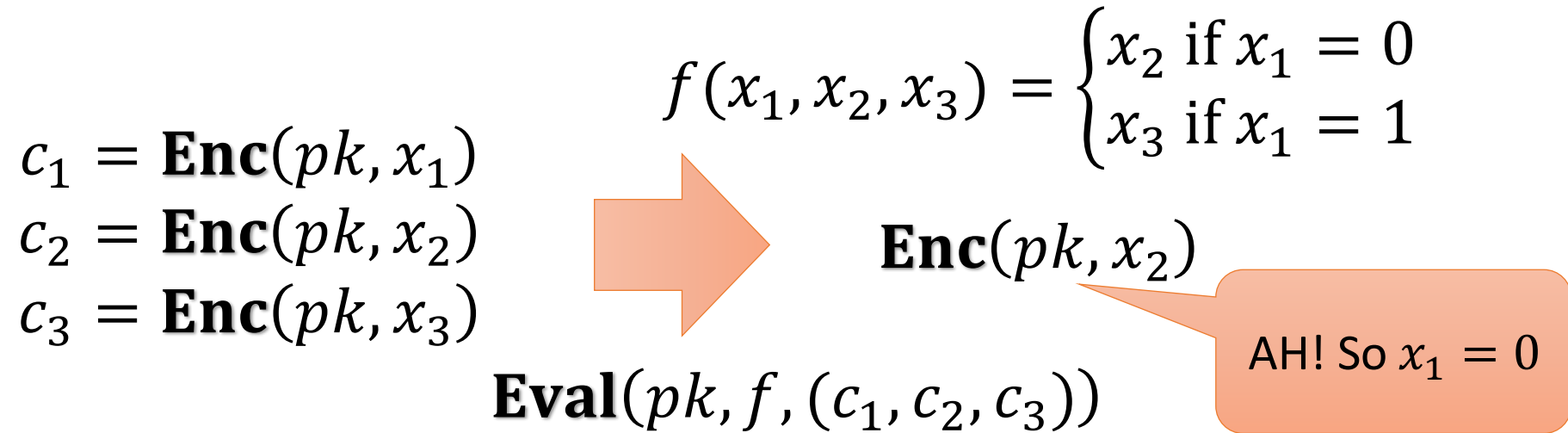
$$\mathbf{Enc}(pk, x) \approx \mathbf{Enc}(pk, 0^{|x|})$$

FHE = Correctness $\forall$ efficient f = Correctness for universal set

Levelled FHE: **Bounded** depth f

- NAND
- $(+, \times)$ over a ring

A Paradox (and its Resolution)



- But remember that encryption is **randomized**!
- Output of **Eval** will look as a **fresh and random** ciphertext

Degrees of Homomorphism

- **Fully-Homomorphic Encryption:** Correctness holds for $\mathcal{C}$ such that $\mathcal{C}_1$ already contains **all** Boolean circuits
 - No need to consider the additional parameter τ
- **Somewhat/Levelled Homomorphic encryption:** Correctness holds for the family $\mathcal{C}$ such that for all $\tau \in \mathbb{N}$ the set $\mathcal{C}_\tau$ contains all Boolean circuits **with depth τ**
- **Additively Homomorphic Encryption:** Correctness holds for $\mathcal{C}$ such that $\mathcal{C}_1$ contains all Boolean circuits **with only XOR gates**
 - No need to consider the additional parameter τ

Trivial FHE?

- Let $(\mathbf{KGen}, \mathbf{Enc}, \mathbf{Dec})$ be **any PKE** scheme
- Define the following **fully-homomorphic** PKE $(\mathbf{KGen}, \mathbf{Enc}, \mathbf{Eval}', \mathbf{Dec}')$:
 - $\mathbf{Eval}'(pk, \Gamma, c) = (\Gamma, c)$
 - $\mathbf{Dec}'(sk, c) = \Gamma(\mathbf{Dec}(sk, c))$

Wish: Complexity of decryption **much less**
than running the circuit from scratch

Strong Homomorphism versus Compactness

- The simplest (and strongest) requirement is to ask that fresh and evaluated ciphertexts **look the same**
- Assume the class $\mathcal{C}$ contains some $\mathcal{C}_{\tau^*}$ which includes AND and XOR gates
 - Evaluate any circuit by evaluating each gate on the outputs of previous gates
 - By **strong homomorphism**, the output distribution is still close to that of a fresh encryption of the output
 - Hence, we have obtained a **strongly fully-homomorphic** PKE!
- A **weaker property** is often **sufficient**: we say that Π is **compact** if there is a **fixed polynomial bound** $B(\cdot)$ on the size of evaluated ciphertexts
 - Here, B **should not depend** on τ or the dependency on τ should be **weak**, say $B(\lambda, \tau) = \text{poly}(\lambda) \cdot o(\log |\mathcal{C}_{\tau}|)$

Secret-Key versus Public-Key FHE

- There is also a **secret-key** variant of FHE
 - Just set $pk = \varepsilon$, and have both **Enc**, **Dec** take only sk as input, whereas **Eval** takes only Γ, c
- Simple transform from SK-FHE to PK-FHE: Given $\Pi = (\mathbf{KGen}, \mathbf{Enc}, \mathbf{Dec}, \mathbf{Eval})$ let $\Pi' = (\mathbf{KGen}', \mathbf{Enc}', \mathbf{Dec}, \mathbf{Eval})$
 - **KGen'** runs **KGen** and lets $pk = (c_0, c_1)$ where $c_0 \leftarrow_{\$} \mathbf{Enc}(sk, 0)$ and $c_1 \leftarrow_{\$} \mathbf{Enc}(sk, 1)$
 - **Enc'**(pk, x) outputs **Eval**(Γ_{id}, c_x) where Γ_{id} represents the identity
 - If Π is **strongly homomorphic**, the output of **Enc'** is **statistically close** to that of **Enc**(sk, x)
 - Both strong homomorphism and semantic security are **preserved**!

The Gentry-Sahai-Waters FHE Scheme

- In what follows we will present the FHE scheme due to:
 - C. Gentry, A. Sahai, B. Waters: "Homomorphic Encryption from Learning with Errors: Conceptually-Simpler, Asymptotically-Faster, Attribute-Based." CRYPTO 2013
- Based on the **Learning with Errors (LWE)** assumption
- Only achieves **levelled homomorphism**
 - But can be **bootstrapped** to **full homomorphism** using a trick by Gentry (under additional assumptions)
- Plaintext space will be $\mathbb{Z}_q = [-q/2, q/2)$, for a large prime q
 - For simplicity let us write $[a]_q$ for $a \bmod q$

Eigenvectors Method (Basic Idea)

- Let $\mathbf{C}_1$ and $\mathbf{C}_2$ be matrices for **eigenvector** $\mathbf{s}$, and **eigenvalues** x_1, x_2 (i.e., $\mathbf{s}^t \cdot \mathbf{C}_i = x_i \cdot \mathbf{s}^t$)
 - $\mathbf{C}_1 + \mathbf{C}_2$ has eigenvalue $x_1 + x_2$ w.r.t. $\mathbf{s}$
 - $\mathbf{C}_1 \cdot \mathbf{C}_2$ has eigenvalue $x_1 \cdot x_2$ w.r.t. $\mathbf{s}$
- Idea: Let $\mathbf{C}$ be the ciphertext, $\mathbf{s}$ be the secret key and x be the plaintext (say over $\mathbb{Z}_q$)
 - Homomorphism for **addition/multiplication**
 - But **insecure**: Easy to compute eigenvalues

Approximate Eigenvectors

- Approximate variant: $\mathbf{s}^t \cdot \mathbf{C} = x \cdot \mathbf{s}^t + \mathbf{e}^t \approx x \cdot \mathbf{s}^t$
 - Decryption **works** as long as $\|\mathbf{e}\|_\infty \ll q$

$$\begin{aligned}\mathbf{s}^t \cdot \mathbf{C}_1 &= x_1 \cdot \mathbf{s}^t + \mathbf{e}_1^t \\ \|\mathbf{e}_1\|_\infty &\ll q\end{aligned}$$

$$\begin{aligned}\mathbf{s}^t \cdot \mathbf{C}_2 &= x_2 \cdot \mathbf{s}^t + \mathbf{e}_2^t \\ \|\mathbf{e}_2\|_\infty &\ll q\end{aligned}$$

- Goal: Define **homomorphic** operations

$$\mathbf{C}_{\text{add}} = \mathbf{C}_1 + \mathbf{C}_2$$

$$\begin{aligned}\mathbf{s}^t \cdot (\mathbf{C}_1 + \mathbf{C}_2) &= \mathbf{s}^t \cdot \mathbf{C}_1 + \mathbf{s}^t \cdot \mathbf{C}_2 \\ &= x_1 \cdot \mathbf{s}^t + \mathbf{e}_1^t + x_2 \cdot \mathbf{s}^t + \mathbf{e}_2^t \\ &= (x_1 + x_2) \cdot \mathbf{s}^t + (\mathbf{e}_1^t + \mathbf{e}_2^t)\end{aligned}$$

Noise **grows** a little!

Approximate Eigenvectors

- Approximate variant: $\mathbf{s}^t \cdot \mathbf{C} = x \cdot \mathbf{s}^t + \mathbf{e}^t \approx x \cdot \mathbf{s}^t$
 - Decryption **works** as long as $\|\mathbf{e}\|_\infty \ll q$

$$\mathbf{s}^t \cdot \mathbf{C}_1 = x_1 \cdot \mathbf{s}^t + \mathbf{e}_1^t$$
$$\|\mathbf{e}_1\|_\infty \ll q$$

$$\mathbf{s}^t \cdot \mathbf{C}_2 = x_2 \cdot \mathbf{s}^t + \mathbf{e}_2^t$$
$$\|\mathbf{e}_2\|_\infty \ll q$$

- Goal: Define **homomorphic** operations

$$\mathbf{C}_{\text{mult}} = \mathbf{C}_1 \cdot \mathbf{C}_2$$

$$\begin{aligned}\mathbf{s}^t \cdot (\mathbf{C}_1 \cdot \mathbf{C}_2) &= (x_1 \cdot \mathbf{s}^t + \mathbf{e}_1^t) \cdot \mathbf{C}_2 \\ &= x_1 \cdot (x_2 \cdot \mathbf{s}^t + \mathbf{e}_2^t) + \mathbf{e}_1^t \cdot \mathbf{C}_2 \\ &= x_1 \cdot x_2 \cdot \mathbf{s}^t + (x_1 \cdot \mathbf{e}_2^t + \mathbf{e}_1^t \cdot \mathbf{C}_2)\end{aligned}$$

Noise **grows**! Needs to be **small**!

Shrinking Gadgets

- Write entries in $\mathbf{C}$ using **binary decomposition**; e.g.

$$\mathbf{C} = \begin{bmatrix} 3 & 5 \\ 1 & 4 \end{bmatrix} \pmod{8} \xrightarrow{\text{yields}} \text{bits}(\mathbf{C}) = \begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 1 & 1 \\ 0 & 1 \\ 0 & 0 \\ 1 & 0 \end{bmatrix} \pmod{8}$$

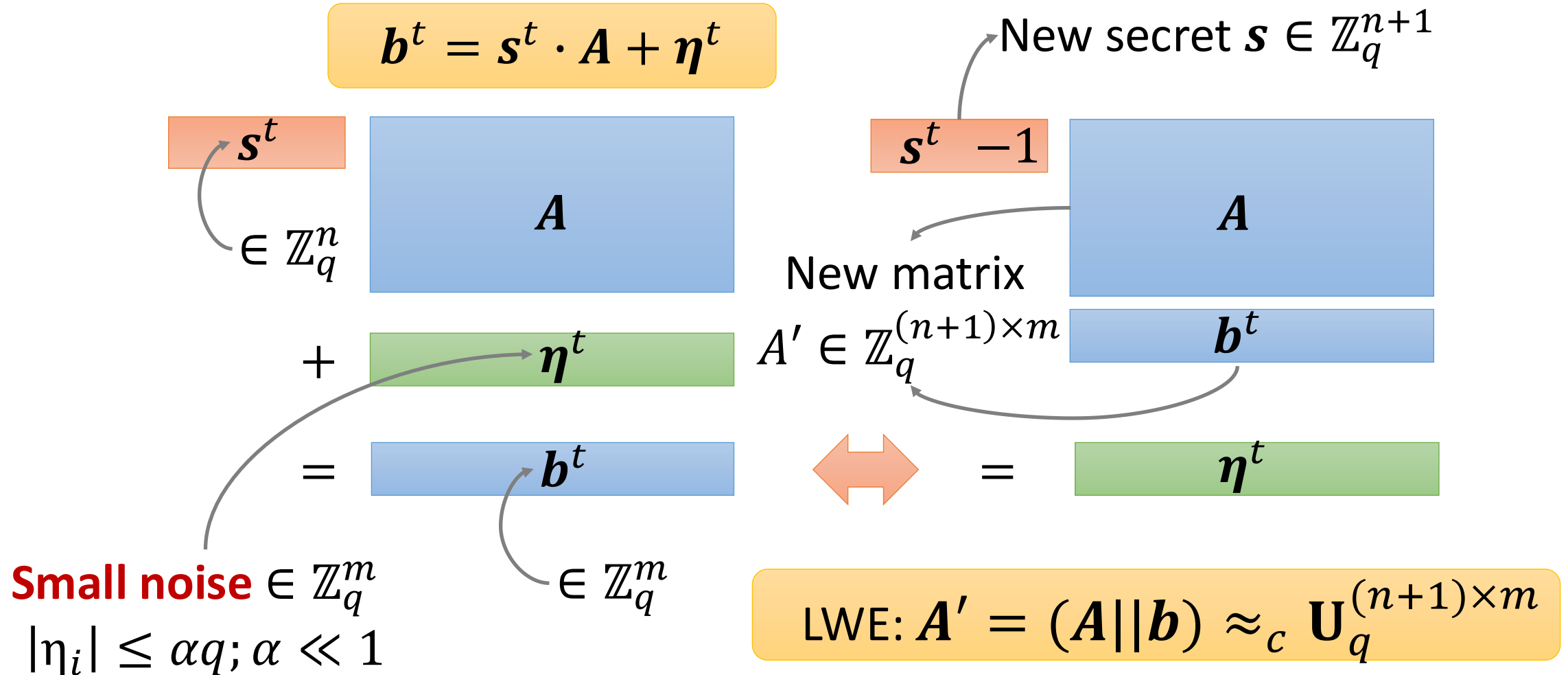
- Reverse** operation:

$$\mathbf{C} = \mathbf{G} \times \mathbf{G}^{-1}(\mathbf{C}) = \begin{bmatrix} 2^{N-1} & \dots & 2 & 1 & 0 & \dots & 0 & 0 \\ 0 & \dots & 0 & 0 & 2^{N-1} & \dots & 2 & 1 \end{bmatrix} \times \text{bits}(\mathbf{C})$$

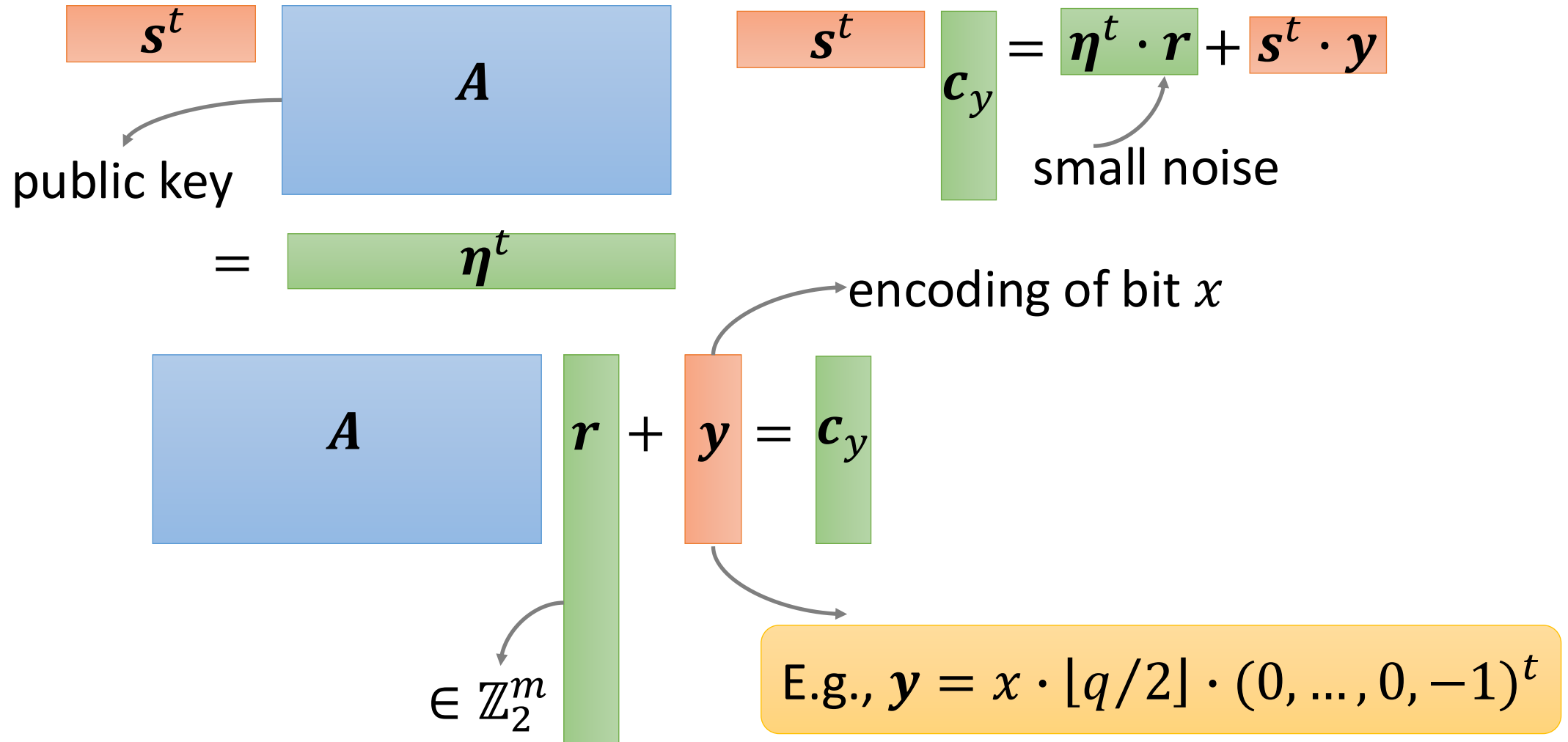
$\xleftrightarrow{k \cdot N = k \lceil \log q \rceil}$

$\Rightarrow \mathbf{s}^t \cdot \mathbf{C} = \mathbf{s}^t \cdot \mathbf{G} \cdot \mathbf{G}^{-1}(\mathbf{C})$

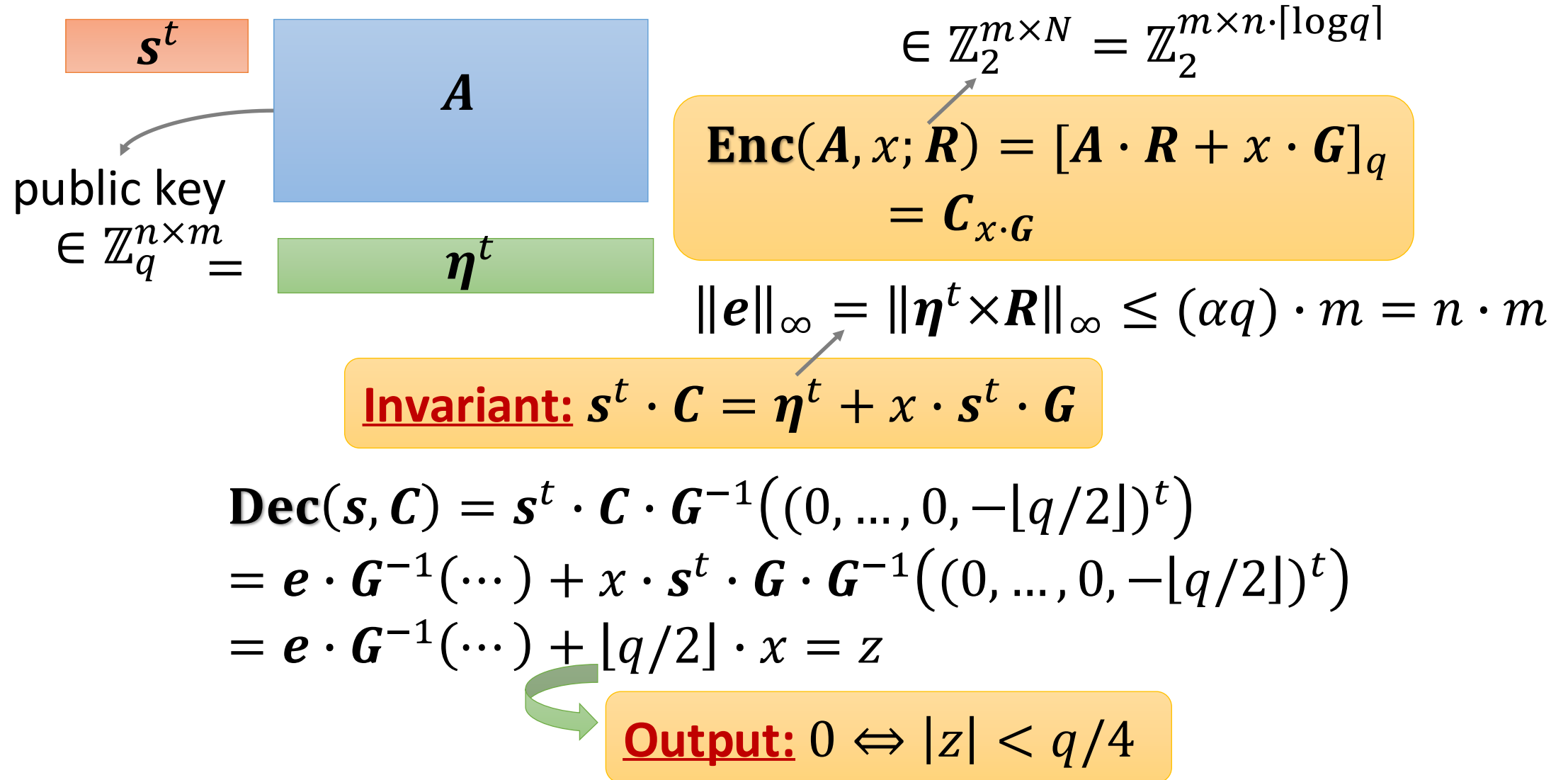
LWE – Rearranging Notation



Regev PKE – Pictorially



The GSW Scheme



The GSW Scheme – Homomorphism

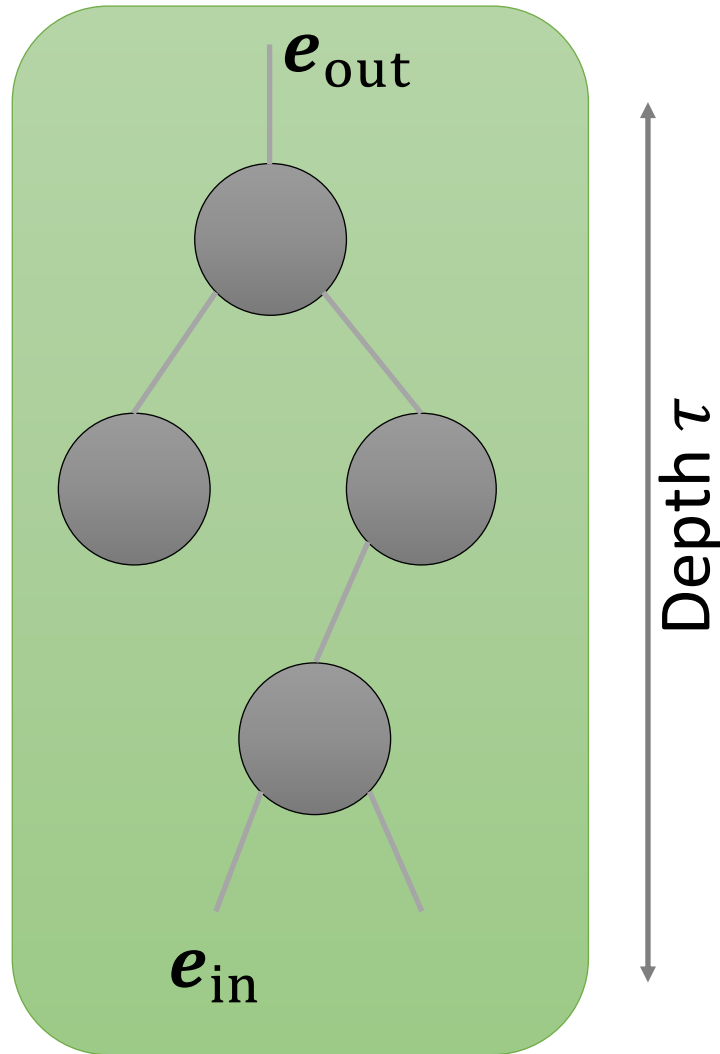
$$\textbf{Invariant: } s^t \cdot \mathcal{C} = e^t + x \cdot s^t \cdot G$$

$$\mathcal{C}_{\text{mult}} = \mathcal{C}_1 \cdot G^{-1}(\mathcal{C}_2)$$

$$\begin{aligned} s^t \cdot \mathcal{C}_1 \cdot G^{-1}(\mathcal{C}_2) &= (e_1^t + x_1 \cdot s^t \cdot G) \cdot G^{-1}(\mathcal{C}_2) \\ &= e_1^t \cdot G^{-1}(\mathcal{C}_2) + x_1 \cdot s^t \cdot G \times G^{-1}(\mathcal{C}_2) \\ &= e_1^t \cdot G^{-1}(\mathcal{C}_2) + x_1 \cdot s^t \cdot \mathcal{C}_2 \\ &= e_1^t \cdot G^{-1}(\mathcal{C}_2) + x_1 \cdot (e_2^t + x_2 \cdot s^t \cdot G) \\ &= (e_1^t \cdot G^{-1}(\mathcal{C}_2) + x_1 \cdot e_2^t) + x_1 x_2 \cdot s^t \cdot G \\ &= e_{\text{mult}} + x_1 x_2 \cdot s^t \cdot G \end{aligned}$$

$$\|e_{\text{mult}}\|_{\infty} \leq N \cdot \|e_1\|_{\infty} + \|e_2\|_{\infty} \leq (N + 1) \cdot \max\{\|e_1\|, \|e_2\|\}$$

The GSW Scheme – Correctness



$$\|e_{out}\|_{\infty} \leq (N + 1)^{\tau+1} m \cdot \alpha q$$

Correctness:

$$n \cdot m \cdot (N + 1)^{\tau+1} < q/4$$

$$\|e_{i+1}\|_{\infty} \leq (N + 1)\|e_i\|_{\infty}$$

$$\|e_{in}\|_{\infty} \leq m \cdot n = m \cdot \alpha q$$

The GSW Scheme – Semantic Security

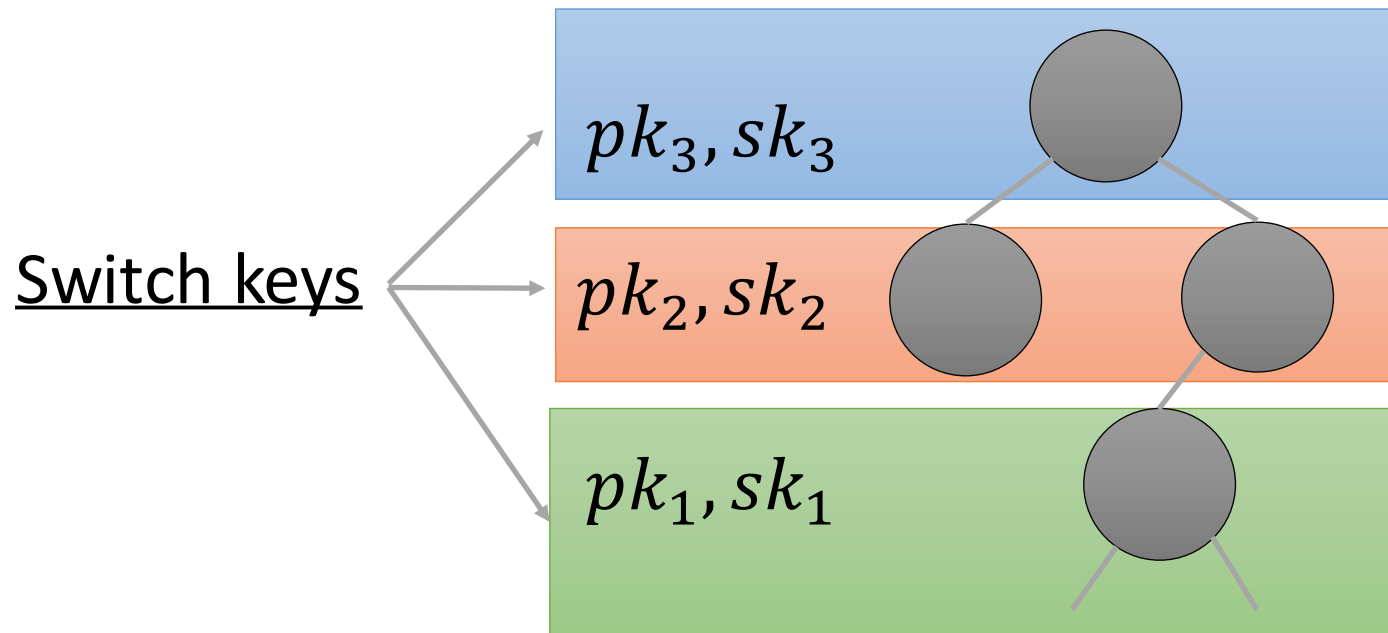
- Similar as in the proof of Regev PKE
- Using the LWE assumption, we move to a **mental experiment** with uniformly random $\mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m}$
- Hence, by the **leftover hash lemma**, with $m = \Theta(n \log q)$, the statistical distance between $(\mathbf{A}, \mathbf{A} \cdot \mathbf{r})$ and uniform is negligible
 - By a **standard argument** over the columns of $\mathbf{R}$, it follows that the statistical distance between $(\mathbf{A}, \mathbf{A} \cdot \mathbf{R})$ and uniform is also negligible
 - Thus, the ciphertext **statistically hides** the plaintext

The GSW Scheme – Parameters

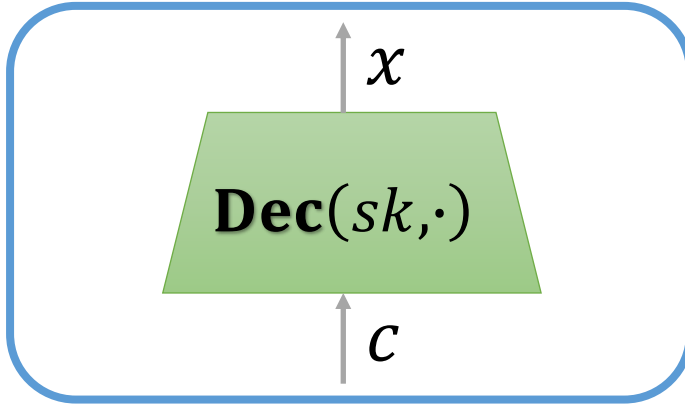
- **Correctness** requires $n \cdot m \cdot (N + 1)^{\tau+1} < q/4$
- **Security** requires $m = \Theta(n \log q)$, e.g., $m \geq 1 + 2n(2 + \log q)$
- **Hardness** of LWE requires $q \leq 2^{n^\epsilon}$ for $\epsilon < 1$
 - Substituting we get $q > (2n \log q)^{\tau+3}$
 - And thus, $n^\epsilon > (\tau + 3)(\log n + \log \log q + 1)$ which for large τ, n yields $n^\epsilon > 2\tau \log n$
 - So, we set $n = \max(\lambda, \lceil 4\tau/\epsilon \log \tau^{1/\epsilon} \rceil)$, $q = \lceil 2^{n^\epsilon} \rceil$, $m = O(n^{1+\epsilon})$, and $\alpha = n/q = n \cdot 2^{-n^\epsilon}$
- Hence, the size of ciphertexts is polynomial in λ, τ thus yielding a **weakly-compact** FHE

Increasing the Homomorphic Capacity

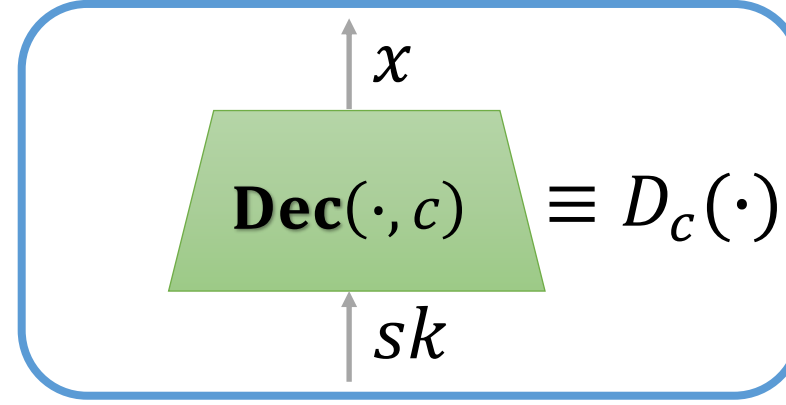
- The only way to increase the homomorphic capacity of GSW is to pick **larger parameters**
- This dependence can be **broken** using a trick by Gentry
- Main idea: Do a few operations, then **switch keys**



How to Switch Keys



Decryption circuit



Dual view

$$\begin{aligned}\mathbf{Eval}_{pk'}(D_c, aux) &= \mathbf{Eval}_{pk'}(D_c, \mathbf{Enc}_{pk'}(sk)) \\ &= \mathbf{Enc}_{pk'}(D_c(sk)) \\ &= \mathbf{Enc}_{pk'}(x)\end{aligned}$$

Bootstrappable Encryption

- Let $W_{\Pi}(\lambda, \tau)$ be the set of all **fresh** and **evaluated** ciphertexts w.r.t. circuits class C_{τ}
 - For all possible keys and all possible inputs to the circuit
- Given $c_1, c_2 \in W_{\Pi}(\lambda, \tau)$, let $D_{c_1, c_2}^*(sk)$ be the **augmented decryption circuit**, defined by

$$D_{c_1, c_2}^*(sk) = \text{NAND}(D_{c_1}(sk), D_{c_2}(sk))$$

- We say that Π is **bootstrappable** if its homomorphic capacity includes all the augmented decryption circuits
 - I.e., $\exists \tau$ s.t. $\forall \lambda \in \mathbb{N}, c_1, c_2 \in W_{\Pi}(\lambda, \tau(\lambda))$, we have $D_{c_1, c_2}^* \in C_{\tau(\lambda)}$

Circular Security

- The above scheme is **compact**, but **not fully homomorphic**, as we need a pair of keys **for each level** in the circuit
- A natural idea is to use a **single pair** (pk, sk) and include in pk' a ciphertext $c^* \leftarrow \mathbf{Enc}(pk, sk)$
 - Correctness still holds for this variant, but the reduction to **semantic security breaks**
- Workaround: Assume **circular security**
 - I.e., $\mathbf{Enc}(pk, 0) \approx_c \mathbf{Enc}(pk, 1)$ even given $c^* \leftarrow \mathbf{Enc}(pk, sk)$
 - GSW is **conjectured** to have this property, but no proof of this fact is known